SubePe

Plan de módulos, contratos y tareas

Sistema de pago de transporte urbano con tarjetas NFC: validador ESP32 con varios canales de comunicación, servidor central y banca del cliente

Proyecto	Curso IoT. Sistema NFC de pago de buses
Versión del documento	1.0 (23/10/2026)
Repositorio	`nfc/` (rama base `main`)
Servidor	`https://subepe.app` (Oracle Cloud)
Alcance	Contexto, decisiones, contratos C1–C5 y módulos M0–M21 divididos en épicas e issues

Este documento basta para implementar sin interpretar: cada issue indica de qué depende, qué archivos toca, los pasos y los criterios de aceptación. Si algo no está claro o no cuadra con el código, se abre una pregunta en el issue y no se improvisa.

Índice

1. Contexto y panorama
2. Decisiones de arquitectura
3. Arquitectura general (diagrama)
4. Convenciones de trabajo
5. Contrato C1: protocolo de sincronización v2
6. Contrato C2: interfaces de enlace del firmware
7. Contrato C3: registro lógico de tarjeta (215 y 424)
8. Contrato C4: API del portal del cliente
9. Contrato C5: interfaz de pasarela de pago
10. Mapa de módulos, instancias y dependencias
11. Instancia 0: M0 contratos en código
12. Instancia 1: módulos independientes (M1–M14)
13. Instancia 2: módulos dependientes (M15–M21)
14. Hardware a comprar
15. Riesgos y temas abiertos
16. Glosario

1. Contexto y panorama

SubePe es un sistema de pago de transporte urbano. Cada pasajero tiene una **tarjeta NFC personal** con datos y saldo. Al subir al bus la acerca al **validador**, que cobra el pasaje en unos cientos de milisegundos y registra el viaje. Los viajes llegan al **servidor central**, que es la fuente de verdad del saldo. La idea es reproducir la experiencia de una tarjeta bancaria con NFC (al estilo BCP) aplicada al pago del bus, incluida una banca del cliente.

1.1 Estado actual (lo que ya funciona)

Parte	Estado
Tarjetas	NTAG215 (familia Ultralight) para pruebas, con formato firmado v2 (`transporte/tarjeta/formato.py`): cabecera firmada, **doble registro de saldo**, contraseña PWD/PACK y contador NFC. 2 MIFARE Classic 1K de prueba. **Objetivo de producción: NTAG 424 DNA** (AES-128), que aún no tenemos.
Programas del lector (PC)	`transporte/` : emisión, validador sin conexión, recargas, bloqueo, con lector USB ACR122U. Sincronizan con `POST /api/sync/` (JSON).
Validador autónomo	`pantalla_esp32/` (entorno `validador`): ESP32-WROOM-32, PN532 por I2C (SDA P21, SCL P22, IRQ P32), LCD SSD1283A (P23, P18, P5, P17, P16), microSD (P26, P27, P14, P13). Cobra con escritura en ~329 ms. Sincroniza por **WiFi** (HTTPS) cada cierto tiempo y tiene portal de configuración de redes.
Servidor	`servidor_api/` : Django 6.1 + DRF + PostgreSQL 17 en Docker y Caddy (HTTPS). App `transporte` (API) y app `panel` (panel de operadores). Sincronización idempotente, detección de fraude (clones, copias restauradas), lista negra, recargas pendientes y tarifas. 25 tests. CI/CD con GitHub Actions y copias de seguridad diarias.
Despliegue	Oracle Cloud. La VM de pago con crédito de prueba vence alrededor del 06/11/2026 y la mudanza a la VM **Always Free** (1 GB de RAM) está planificada para el 1–3/11.

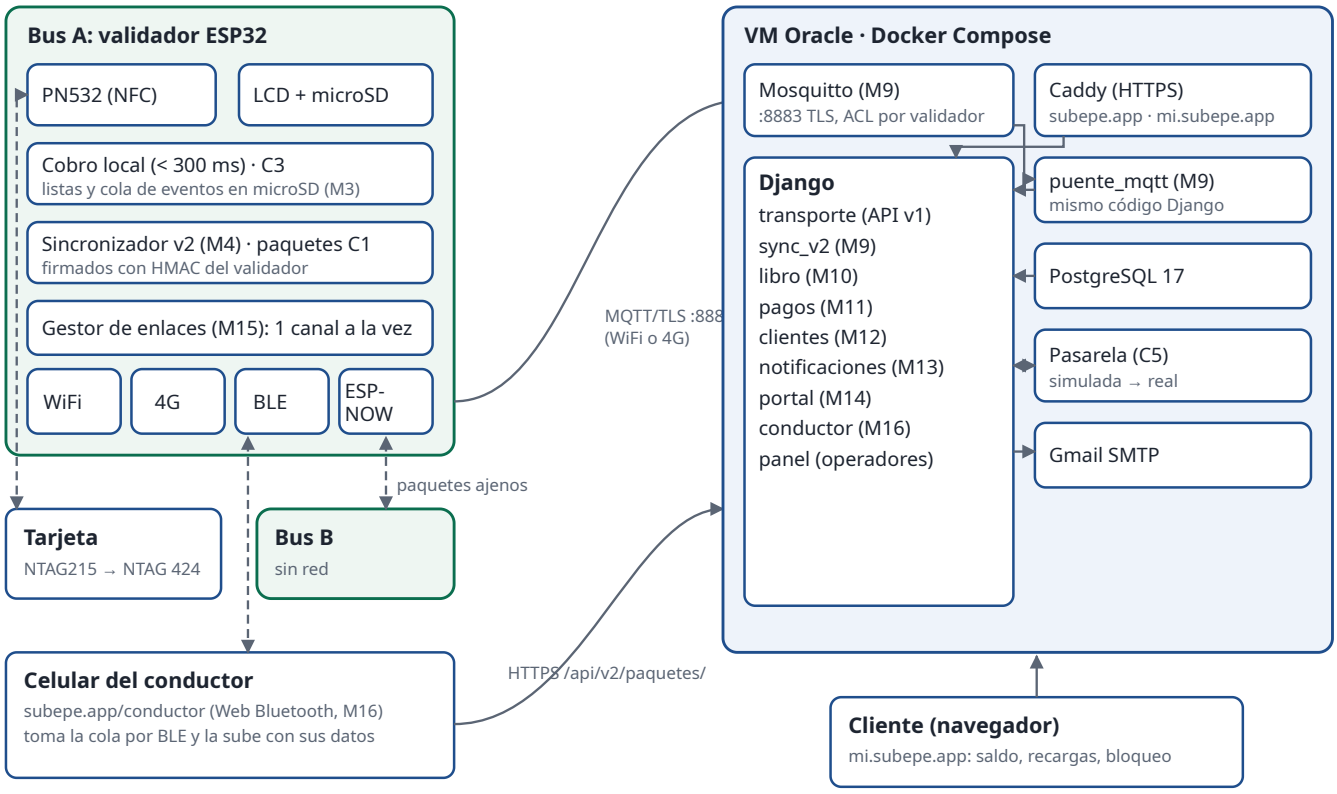
1.2 Restricciones

- **Presupuesto cero en infraestructura.** Solo capas gratuitas y nada que exija otro despliegue: todo lo nuevo vive en el mismo Docker Compose de la VM. La única compra prevista es el **módulo celular** (sección 14).
- **Tarjetas:** se trabaja con NTAG215 hasta tener NTAG 424 DNA. El sistema debe soportar las dos con **un mismo registro lógico** (C3); al cambiar de tarjeta solo cambia el controlador.
- **Conectividad no garantizada:** no se sabe si el conductor compartirá datos y no hay WiFi en toda la ruta. El validador **debe cobrar sin red** y sincronizar por el canal que haya.
- **Dinero real:** aunque la pasarela de pago es simulada, todo lo demás (libro contable, idempotencia, conciliación, auditoría) se construye como en producción.
- Idioma: código, comentarios, commits e interfaz en **español**.

2. Decisiones de arquitectura

#	Decisión	Motivo
D1	**Saldo híbrido**: el servidor es la verdad y la tarjeta lleva una copia protegida	Permite cobrar sin conexión. Con la 424 la copia pasa a ser confiable porque solo se escribe con clave AES. Se reutiliza todo lo construido.
D2	**La validación siempre es local** (< 300 ms) y la red solo propaga datos	Ninguna red garantiza latencia ni cobertura. Retrasos aceptados: bloqueos 1–5 min, recargas 5–15 min, viajes horas, tarifas 1 día.
D3	**Varios canales independientes**: 4G, WiFi (router o hotspot), celular del conductor por BLE y ESP-NOW entre buses	Robustez. Un **gestor de enlaces** usa un solo canal de subida a la vez, por prioridad, sin solaparlos. ESP-NOW es un intercambio entre buses aparte.
D4	**Paquetes firmados de extremo a extremo** (C1), sin importar el canal	El celular del conductor o **otro bus** pueden transportar datos sin ser de confianza: el validador firma con HMAC y el servidor con ECDSA P-256. Así un bus sin red sube sus viajes a través de otro.
D5	**MQTT** (Mosquitto) sobre TLS como transporte principal de 4G y WiFi	Es un estándar abierto (OASIS, ISO/IEC 20922) y Mosquitto es software libre (Eclipse). La conexión persistente y el envío de bloqueos desde el servidor gastan < 1 MB/día por bus, frente a ~15–20 MB/día del sondeo HTTPS actual.
D6	Fiabilidad con **acuses de la aplicación**, no con QoS de MQTT	El mismo acuse (paquete 0x81) sirve por MQTT, BLE, HTTP y ESP-NOW. MQTT se usa con QoS 0 y la librería simple PubSubClient alcanza.
D7	**Banca del cliente** en el subdominio `mi.subepe.app`, dentro del mismo Django	No requiere otro despliegue (solo un registro DNS y dos líneas en el `Caddyfile`). Las cookies de sesión del cliente quedan separadas de las del panel. API JSON (C4) para una futura app.
D8	**Alta bancaria**: la tarjeta se emite en ventanilla con DNI y el portal se activa con un código de un solo uso	Una tarjeta perdida en el camino no da acceso al portal. Igual que con las tarjetas bancarias.
D9	**Libro mayor de doble partida** inmutable; pasarela detrás de una interfaz (C5) con un adaptador simulado	Se trata dinero real: cada sol se traza y concilia. Cambiar a una pasarela real es escribir un adaptador.
D10	**Correo por Gmail SMTP** (contraseña de aplicación) para OTP y avisos	Es gratis (~500 correos/día). Sirve como segundo canal para recuperar la cuenta y detectar robos. Más adelante se puede pasar a Brevo o Resend con `@subepe.app`.
D11	El relé por celular lo usa **solo el conductor**, con una **página web** (Web Bluetooth), sin app	Sin instalación. Funciona en Chrome para Android. Los paquetes van firmados, así que la página no puede alterarlos.

3. Arquitectura general



> **Lectura del diagrama:** el bus cobra solo. El sincronizador produce paquetes C1 firmados y el gestor los envía por el canal disponible. Si el bus A no tiene red, entrega sus paquetes al conductor por BLE o a otro bus por ESP-NOW, y quien tenga red los sube. El servidor verifica la firma del bus de origen sin importar por dónde llegaron.

4. Convenciones de trabajo

4.1 Estructura de los issues

- **Épica** = módulo (`M9`). **Issue** = tarea entregable (`M9-03`), pensada para 0,5–3 días de trabajo.
- Etiquetas de GitHub: `epica:M9`, `area:firmware|servidor|portal|tarjeta|hardware|docs`, `instancia:0|1|2`.
- Título del issue: `[M9-03] Servicio procesar_paquete`. El cuerpo copia la ficha de este documento (depende de, archivos, pasos, criterios).
- Si el issue necesita algo que no está en un contrato, **no se inventa**: se comenta en el issue, se actualiza el contrato y se sube su versión.

4.2 Ramas, commits y PR

- Rama: `tarea/<ID>-<descripcion-corta>`, por ejemplo `tarea/M9-03-procesar-paquete`. Se sale de `main` y se vuelve a `main` por PR.
- Commits en español, en imperativo o descriptivos (como los actuales: "Validador ESP32 autónomo con lector PN532 por I2C"). Un PR por issue.
- El PR enlaza el issue (`Cierra #N`) y describe cómo se probó.

4.3 Definición de terminado (todas las tareas)

- Se cumplen todos los criterios de aceptación del issue.
- Servidor: `docker compose exec web python manage.py test` pasa, y los tests nuevos cubren los casos del issue.
- Firmware: `pio run -e validador` compila sin advertencias nuevas; `pio test` pasa cuando hay tests; se probó en la placa si el issue toca hardware.
- La documentación afectada está al día (`README.md` , `pantalla_esp32/README.md` , contrato).
- Ningún secreto en el repositorio: las claves van en `.env` , NVS o `transporte/claves/` , que están en `.gitignore` .
- Montos **siempre en céntimos enteros**; nunca `float` .

5. Contrato C1: protocolo de sincronización v2

Define los bytes que viajan entre el validador y el servidor por **cualquier** canal (MQTT, HTTP, BLE, ESP-NOW). Formato binario de tamaño fijo, **little-endian** (en Python: `struct` con `"<"`). Convive con la API v1 (`/api/sync/`), que no se toca mientras los validadores PC la usen.

5.1 Cabecera común (20 bytes)

Offset	Bytes	Campo	Descripción
0	2	magia	`0x53 0x50` ("SP")
2	1	versión	`1`
3	1	tipo	Ver 5.2
4	2	origen	N.º de validador que creó el paquete; `0` = servidor
6	2	destino	N.º de validador; `0` = servidor; `0xFFFF` = todos
8	4	secuencia	uint32 creciente por emisor; nunca se repite (persistida)
12	4	creado	Hora unix (s) de creación en el emisor
16	2	largo	Bytes del cuerpo (sin cabecera ni firma)
18	2	reservado	`0`

Después del cuerpo va la **firma**, calculada sobre `cabecera || cuerpo`:

- Si `origen ≠ 0` (validador): `HMAC-SHA256(K_val, cabecera||cuerpo)` truncado a los **16 primeros bytes**. `K_val` son 32 bytes aleatorios propios de cada validador, generados por el servidor (M0-06) y guardados en el campo `Validador.clave_hmac` y en la NVS de la ESP32.
- Si `origen = 0` (servidor): **ECDSA P-256 con SHA-256**; firma de 64 bytes (`r || s`, 32 bytes cada uno, big-endian). La clave privada está en `.env` (`LISTAS_CLAVE_PRIVADA`, PEM) y la pública va compilada en el firmware (`clave_servidor.h`).

Tamaño total = 20 + largo + (16 o 64). Un paquete mide como máximo **2048 bytes**. Si la magia, la versión, el largo o la firma no son válidos, el paquete **se descarta** y se registra; nunca se procesa a medias.

5.2 Tipos de paquete

Tipo	Nombre	Sentido	Cuerpo
`0x01`	EVENTOS	validador → servidor	`n` (uint8, 1–30) + 3 bytes reservados + `n` × evento de 64 B (5.3)
`0x02`	HOLA	validador → servidor	32 B (5.4). Se envía al conectar y cada 5 min
`0x81`	ACUSE	servidor → validador	`hasta_evento` uint32 + `hora_servidor` uint32 (8 B)
`0x82`	LISTAS	servidor → validador / todos	Cambios de lista negra y recargas (5.5)
`0x83`	TARIFAS	servidor → validador	Tabla completa de tarifas (5.6)

5.3 Evento (64 bytes)

Offset	Bytes	Campo	Equivale en la API v1 a
0	4	id_evento (uint32, correlativo del validador)	`id`
4	1	tipo: 1 viaje, 2 incidencia, 3 fraude	`tipo`
5	1	largo del UID (4 o 7)	—
6	2	operación (uint16)	`operacion`
8	4	fecha (unix, s)	`fecha`
12	4	id_tarjeta (uint32)	`tarjeta_id`

16	10	UID (bytes, con ceros de relleno)	`uid` (en hex en v1)
26	2	reservado = 0	—
28	4	monto (int32, céntimos; 0 si no aplica)	`monto`
32	4	saldo_final (int32)	`saldo_final`
36	4	contador NFC (uint32; `0xFFFFFFFF` = no hay)	`contador`
40	4	recarga_hasta (uint32; 0 = ninguna)	`recarga_hasta`
44	20	detalle ASCII, relleno con 0 (texto de la incidencia truncado)	`detalle`

El servidor convierte cada evento al `dict` que ya acepta `servicios.procesar_evento()`, así que la lógica de fraude, bloqueo y recargas **no se duplica**.

5.4 HOLA (32 bytes)

Offset	Bytes	Campo
0	4	version_listas: última versión de LISTAS aplicada
4	4	version_tarifas: versión de TARIFAS guardada
8	4	ultimo_evento: último id_evento creado
12	2	pendientes: eventos propios sin acuse
14	2	ajenos: paquetes de otros buses guardados
16	1	enlace: 1 WiFi, 2 4G, 3 BLE, 4 reenviado por otro bus
17	1	señal (int8, dBm/2; 0 = no disponible)
18	2	reservado
20	4	bytes_tx_hoy (todos los canales)
24	4	bytes_rx_hoy
28	4	version_firmware (uint32, AAAAMMDD de compilación)

5.5 LISTAS (cambios versionados)

Cuerpo: `version_desde` uint32 · `version_hasta` uint32 · `n` uint16 (0–120) · `urgente` uint8 · reservado uint8 · `n` × entrada de 16 B:

Offset	Bytes	Campo	Tipo 1: BLOQUEO	Tipo 2: RECARGA
0	1	tipo	1	2
1	1	op	1 = agregar a la lista negra, 2 = quitar	1 = pendiente, 2 = ya aplicada (borrar)
2	2	reservado	0	0
4	4	a	id_tarjeta	id_cuenta
8	4	b	0	seq de la recarga
12	4	c	0	monto (céntimos)

Reglas (las cumplen el servidor y el validador):

1. Cada cambio de lista en el servidor recibe una **versión global** creciente (tabla `CambioLista`, M9-01).
2. El validador aplica un paquete solo si `version_desde == su version_listas`, o si `version_desde == 0`: en ese caso borra sus listas y las reconstruye. Después guarda `version_listas = version_hasta`.
3. Si el servidor ve que un validador está atrasado más de 2000 versiones, le manda una **foto completa**: primer paquete con `version_desde = 0` y los siguientes encadenados, solo con `op = 1` del estado actual.
4. Un bloqueo nuevo se publica además como paquete **urgente** (`urgente = 1`, `destino = 0xFFFF`, 1 entrada). El validador aplica las entradas urgentes de BLOQUEO **aunque no sean contiguas**, sin cambiar su version_listas; cuando le llegue el cambio en orden, aplicarlo de nuevo no tiene efecto.

5. Un paquete con firma inválida se descarta y se registra como incidencia "lista con firma inválida".

5.6 TARIFAS

Cuerpo: `version` uint32 (= marca unix de la última modificación de cualquier tarifa) · `n` uint8 · 3 reservados · `n` × 24 B: `codigo` uint8 · 3 reservados · `precio` uint32 (céntimos) · `nombre` 16 B ASCII con relleno de ceros. El servidor lo envía cuando la `version_tarifas` del HOLA es distinta de la actual.

5.7 Acuses

- El servidor guarda, por validador, `ultimo_evento_contiguo`: el mayor id N tal que recibió todos los ids de 1 a N.
- ACUSE.`hasta_evento` = ese valor. El validador borra de su cola los eventos con id ≤ hasta_evento. Como los acuses van **firmados por el servidor**, un intermediario no puede hacer que el validador borre eventos sin subir.
- Reenviar un evento ya recibido no hace nada (idempotencia por `(validador, id_evento)`).
- Al recibir un HOLA, el servidor responde siempre ACUSE + LISTAS pendientes + TARIFAS (si cambiaron).

5.8 Transporte por MQTT

Tópico	Publica	Contenido
`sp/v/{n}/sube`	validador n	Paquetes 0x01 y 0x02 propios **y ajenos** (de otros buses), uno por mensaje
`sp/v/{n}/baja`	puente del servidor	Respuestas para el validador n y acuses de los paquetes ajenos que subió n
`sp/todos/urgente`	puente del servidor	LISTAS urgentes (bloqueos)
`sp/v/{n}/conexion`	validador n (y su *last will*)	`"1"` al conectar y `"0"` como *last will*, con *retain*

Broker: Mosquitto 2.x, puerto **8883** (TLS con el certificado de Let's Encrypt de Caddy). Usuario MQTT = número de validador; contraseña = token MQTT propio. Se usa **QoS 0**, sin sesión persistente y con *keepalive* de 60 s. ACL:

```
user puente
topic readwrite sp/#

pattern write sp/v/%u/sube
pattern write sp/v/%u/conexion
pattern read  sp/v/%u/baja
topic read    sp/todos/#
```

5.9 Transporte por HTTP (celular del conductor y respaldo)

`POST https://subepe.app/api/v2/paquetes/` · `Content-Type: application/octet-stream` · cuerpo = paquetes C1 concatenados (cada uno se delimita con su campo `largo`). Se autentica con una sesión de usuario del grupo `conductores` o con el token DRF de un validador. La respuesta (`200`, octet-stream) son los paquetes de respuesta concatenados. Error `400` si el cuerpo supera 64 KB.

5.10 Fragmentación (BLE y ESP-NOW)

Los dos canales usan la **misma trama**, de máximo 240 bytes:

Offset	Bytes	Campo
0	2	magia `"EN"`
2	1	clase: 1 BALIZA, 2 FRAGMENTO, 3 PEDIDO
3	1	reservado
4	2	validador emisor (0 = celular o servidor)

6	…	según la clase (abajo)

- **FRAGMENTO**: `id_mensaje` uint32 · `indice` uint8 · `total` uint8 (≤ 10) · `largo` uint8 · datos (≤ 220 B). Un mensaje es un paquete C1 completo. Se reensambla con un tiempo límite de 3 s; si falta un fragmento, se descarta el mensaje entero.
- **BALIZA** (solo ESP-NOW, a difusión cada 2 s ± 500 ms al azar): `version_listas` u32 · `version_tarifas` u32 · `pendientes` u16 · `ajenos` u16 · `flags` u8 (bit0 = tiene canal con el servidor ahora; bit1 = acepta ajenos).
- **PEDIDO**: `que` u8 (1 = LISTAS desde versión X, 2 = TARIFAS, 3 = acuses para mí) · `x` u32.

UUIDs BLE (GATT): servicio `a3f1c000-5ab0-4e50-9e2b-53554245504e`. Características: `…c001…` **info** (read: cuerpo HOLA + n.º de validador, 34 B), `…c002…` **salida** (notify: tramas FRAGMENTO del validador al celular), `…c003…` **entrada** (write with response: tramas FRAGMENTO del celular al validador), `…c004…` **control** (write: 1 = empezar a enviar la cola, 2 = parar, 3 = fin de sesión). En todas solo cambian los dígitos `c00X` del primer grupo.

5.11 Vectores de prueba

`docs/contratos/vectores_c1.json` (M0-01) contiene paquetes de ejemplo con claves de prueba fijas y su salida en hex. Las dos bibliotecas (Python y C++) **deben** producir y aceptar exactamente esos bytes. Es la garantía de que firmware y servidor se entienden.

6. Contrato C2: interfaces de enlace del firmware

Carpeta `pantalla_esp32/lib/Enlaces/`. Hay dos tipos de canal:

- **Enlace IP** (WiFi, 4G): da un flujo TLS hacia el servidor, por el que corre MQTT.
- **Canal de paquetes** (MQTT sobre un enlace IP, o BLE): mueve paquetes C1 completos. El sincronizador (M4) solo conoce `CanalPaquetes`.

```
// Enlace.h — enlace IP hacia internet (WiFi o 4G)
enum class EstadoEnlace : uint8_t { Apagado, Conectando, Listo, Error };

class Enlace {
 public:
  virtual ~Enlace() = default;
  virtual const char *nombre() const = 0;      // "wifi" | "4g"
  virtual uint8_t codigo() const = 0;          // 1 wifi, 2 4g (campo enlace de HOLA)
  virtual void encender() = 0;                 // empieza a conectar sin bloquear
  virtual void apagar() = 0;                   // libera radio/módem
  virtual void atender() = 0;                  // en cada loop(); nunca bloquea más de 50 ms
  virtual EstadoEnlace estado() const = 0;
  virtual int8_t senal() const = 0;            // dBm/2, 0 si no se sabe
  // Cliente TLS ya conectado a host:puerto (verifica con raices_tls.h).
  // nullptr si el enlace no está Listo o falla la conexión.
  virtual Client *abrir(const char *host, uint16_t puerto) = 0;
  virtual void cerrar() = 0;
  virtual uint32_t bytesTx() const = 0;        // desde el arranque
  virtual uint32_t bytesRx() const = 0;
};

// CanalPaquetes.h — mueve paquetes C1 completos
class CanalPaquetes {
 public:
  virtual ~CanalPaquetes() = default;
  virtual const char *nombre() const = 0;      // "mqtt-wifi" | "mqtt-4g" | "ble"
  virtual uint8_t codigoEnlace() const = 0;    // 1, 2 o 3
  virtual bool listo() const = 0;              // puede enviar ahora
  virtual void atender() = 0;                  // en cada loop()
  virtual bool enviar(const uint8_t *paquete, size_t largo) = 0;
  // Paquete recibido del servidor (copiado en buf); devuelve su largo o 0 si no hay
  virtual size_t recibir(uint8_t *buf, size_t max) = 0;
};

// Vecinos.h — ESP-NOW entre buses (no es canal de subida)
class Vecinos {
 public:
  virtual void iniciar(uint8_t canalWifi) = 0; // 1 por defecto
  virtual void atender() = 0;
  virtual void pausar(bool si) = 0;            // cuando el WiFi STA usa otro canal
  virtual size_t cantidad() const = 0;         // vecinos vistos en los últimos 10 s
};
```

Implementaciones previstas: `EnlaceWifi` (M6), `Enlace4G` (M5), `CanalMqtt(Enlace&)` (M15), `CanalBle` (M7), `VecinosEspNow` (M8). Para pruebas: `EnlaceFalso` y `CanalFalso` (M0-03).

Reglas del gestor (las implementa M15, pero los módulos las asumen desde ya):

1. Hay **un solo canal de subida activo** a la vez. Prioridad: **WiFi (0) > 4G (1) > BLE (2)**. WiFi va primero porque no consume el plan de datos.
2. Un canal de mayor prioridad que pasa a Listo toma el control **después** del intercambio en curso y tras 10 s estable (histéresis). Nunca se corta un paquete a medias.

3. Con WiFi Listo durante más de 5 min, el 4G cierra MQTT y queda registrado en la red sin tráfico. Si el WiFi se cae, el 4G retoma.
4. BLE solo se activa cuando hay un celular conectado que pide la cola; mientras dure esa sesión es el canal activo si no hay WiFi ni 4G.
5. ESP-NOW funciona en paralelo, en el canal de radio 1. Si el WiFi STA está conectado a un AP en otro canal, ESP-NOW se pausa.

7. Contrato C3: registro lógico de tarjeta (NTAG215 y NTAG 424)

Decisión: los **72 bytes** del área del sistema (cabecera firmada de 24 B en los offsets 0–23, reservado 24–31, registro A en 32–51 y registro B en 52–71; formato v2 de `transporte/tarjeta/formato.py`) son **idénticos en las dos tarjetas**. Solo cambia dónde se guardan y cómo se protegen. `formato.py` y `Formato.cpp` no cambian.

	NTAG215 (ahora)	NTAG 424 DNA (producción)
Ubicación del área	Páginas 8–25	Archivo estándar **03** (128 B), offsets 0–71
Protección de escritura	PWD/PACK desde la página 8	Escritura solo con **K2**, CommMode **Full** (cifrado + MAC)
Protección de lectura	Libre (los datos van firmados)	Lectura con **K1**, CommMode Full
Firma HMAC de los registros	Obligatoria	Se mantiene (defensa en profundidad; el formato es el mismo)
Campo "contador"	Contador NFC (READ_CNT)	`SDMReadCtr` del archivo 02 (GetFileCounters)
Anti-clonación	En el servidor (operación o contador repetidos)	Autenticación AES mutua (AuthenticateEV2First); el servidor sigue vigilando
Consulta con el celular	No	Archivo 02 NDEF con **SUN**: `https://mi.subepe.app/t?p=<PICCData>&m=<CMAC>`

Claves de la NTAG 424 (5 ranuras)

Ranura	Uso	Diversificada por UID
K0	Clave maestra de la aplicación: cambiar claves y configuración de archivos	Sí
K1	Leer el archivo 03	Sí
K2	Escribir el archivo 03	Sí
K3	SUN: cifrar PICCData (UID + contador). **Común a todas las tarjetas**, porque el servidor no conoce el UID antes de descifrar	No
K4	SUN: CMAC del mensaje	Sí

Diversificación: **NXP AN10922, AES-128**: `K_i(UID) = AES-CMAC(Kmaestra_i, 0x01 || UID(7) || "SubePe")` con el relleno que define AN10922. Las claves maestras viven en el servidor (`.env`) y en el validador (NVS, M2).
Interfaz del controlador (Python y C++, mismos nombres):

```
class TarjetaFisica(Protocol):          # transporte/tarjeta/base.py
    uid: bytes                           # 7 bytes
    def autenticar(self) -> None: ...    # 215: PWD_AUTH + PACK | 424: AuthenticateEV2First(K1/K2)
    def leer_area(self) -> bytes: ...    # exactamente 72 bytes
    def escribir_cabecera(self, datos: bytes) -> None: ...          # 24 bytes, offset 0
    def escribir_registro(self, indice: int, datos: bytes) -> None: # indice 0 = A (offset 32), 1
= B (offset 52); 20 bytes
    def contador(self) -> int | None: ...
```

8. Contrato C4: API del portal del cliente

Base: `https://mi.subepe.app/api/v1/`. JSON en UTF-8. Autenticación por **cookie de sesión** del host `mi.subepe.app` más cabecera `X-CSRFToken` en los métodos que modifican datos. Montos en **céntimos** (int). Fechas en ISO 8601 con zona horaria (`2026-10-23T08:15:00-05:00`). El archivo fuente es `docs/contratos/api-cliente-v1.yaml` (OpenAPI 3.1, M0-04), y esta tabla es su resumen.

Errores

Siempre `{"error": {"codigo": "...", "mensaje": "texto para mostrar", "campos": {"campo": ["..."]}}}`. Códigos: `no_autenticado` (401), `prohibido` (403), `no_encontrado` (404), `validacion` (400), `credenciales_invalidas` (401), `bloqueo_temporal` (429, con `reintentar_en` en segundos), `otp_invalido` (400), `otp_expirado` (400), `activacion_invalida` (400), `monto_fuera_de_limite` (400), `limite_diario` (400), `idempotencia_conflicto` (409), `tarjeta_no_activa` (409).

Método y ruta	Entrada	Salida (200/201) y notas
Activación (sin sesión)		
`POST activacion/`	`documento`, `numero_tarjeta` (9 dígitos impresos), `codigo` (8 caracteres)	`{token_activacion, nombre_enmascarado}`. El token es válido 15 min. 5 intentos fallidos por documento → `bloqueo_temporal` por 30 min
`POST activacion/correo/`	`token_activacion`, `correo`	`{enviado: true}`. Envía un OTP de 6 dígitos válido 10 min
`POST activacion/confirmar/`	`token_activacion`, `otp`, `contrasena`	`201 {ok: true}`. Crea el usuario, deja el código de activación como usado e inicia sesión
Sesión		
`POST sesion/`	`documento`, `contrasena`	`{requiere_otp: bool}`. Si el dispositivo es nuevo, envía un OTP por correo y la sesión queda a medias
`POST sesion/otp/`	`otp`, `recordar_dispositivo` (bool)	`{ok: true}`. "Recordar" deja el dispositivo como confiable 90 días
`DELETE sesion/`	—	`204`
`POST contrasena/recuperar/`	`documento`	Siempre `202`, exista o no el documento; manda un enlace por correo (válido 30 min)
`POST contrasena/restablecer/`	`uid`, `token`, `contrasena`	`{ok: true}`. Cierra todas las sesiones
`POST contrasena/cambiar/`	`actual`, `nueva`	`{ok: true}`. Cierra las demás sesiones
Consulta (con sesión)		
`GET resumen/`	—	`{cliente:{nombre, documento_enmascarado, correo}, cuenta:{id, saldo, por_cargar_en_tarjeta, tarifa:{codigo, nombre, precio}}, tarjetas:[{id, numero_impreso, estado, emitida, ultimo_uso}]}`. `por_cargar_en_tarjeta` = suma de las recargas que aún no llegan a la tarjeta
`GET movimientos/?desde=&hasta=&tipo=&pagina=`	`tipo` ∈ viaje, recarga, emision, ajuste	`{resultados:[{fecha, tipo, monto, saldo_final, descripcion, validador}], pagina, paginas}`. 20 por página

`GET perfil/`	—	`{nombre, documento_enmascarado, correo, creado}`
`POST perfil/correo/` · `POST perfil/correo/confirmar/`	`correo` · `otp`	Cambia el correo solo después de verificar el OTP enviado al correo nuevo; avisa al anterior
`GET seguridad/accesos/`	—	Últimos 20: `[{fecha, ip, dispositivo, exito}]`
`GET seguridad/dispositivos/` · `DELETE seguridad/dispositivos/{id}/`	—	`[{id, nombre, ultimo_uso, confiable_hasta}]` · `204`
Tarjeta		
`POST tarjetas/{id}/bloquear/`	`motivo` ∈ robo, perdida; `contrasena`	`{estado:"bloqueada", bloqueada_en}`. Irreversible: una tarjeta bloqueada no se reactiva y se emite otra
`POST reposiciones/` · `GET reposiciones/`	`tarjeta_id`	`{id, estado: solicitada|lista|entregada, creada}`. Solo si la tarjeta está bloqueada
Recargas		
`POST recargas/ordenes/`	Cabecera `Idempotency-Key` (UUID v4); `monto` (100–20000)	`201 {id, estado, monto, url_pago, expira}`. Repetir la misma clave devuelve la misma orden; la misma clave con otro monto da `409`
`GET recargas/ordenes/` · `GET recargas/ordenes/{id}/`	—	`{id, estado, monto, creada, pagada_en, acreditada_en}`. Estados: creada, pendiente, pagada, acreditada, fallida, expirada, reembolsada

9. Contrato C5: interfaz de pasarela de pago

Archivo `servidor_api/pagos/pasarelas/base.py`. El adaptador activo se elige con el ajuste `PAGOS_PASARELA` (por ahora `"simulada"`).

```
@dataclass(frozen=True)
class CobroCreado:      id_externo: str; url_pago: str; expira: datetime
@dataclass(frozen=True)
class EstadoCobro:      id_externo: str; estado: str  # "pendiente"|"pagado"|"fallido"|"expirado"
                        monto: int; pagado_en: datetime | None
@dataclass(frozen=True)
class EventoPasarela:   id_evento: str; tipo: str     #
"cobro.pagado"|"cobro.fallido"|"reembolso.hecho"
                        id_externo: str
@dataclass(frozen=True)
class Transaccion:      id_externo: str; monto: int; estado: str; fecha: datetime

class FirmaInvalida(Exception): ...
class ErrorPasarela(Exception): ...       # red, timeout o respuesta inesperada: se reintenta

class Pasarela(Protocol):
    nombre: str
    def crear_cobro(self, *, orden_id: UUID, monto: int, descripcion: str,
                    correo: str, url_retorno: str) -> CobroCreado: ...
    def consultar(self, id_externo: str) -> EstadoCobro: ...
    def verificar_webhook(self, cabeceras: Mapping[str, str], cuerpo: bytes) -> EventoPasarela:
...
    def reembolsar(self, id_externo: str, monto: int) -> str: ...      # id del reembolso
    def listar_transacciones(self, dia: date) -> list[Transaccion]: ...
```

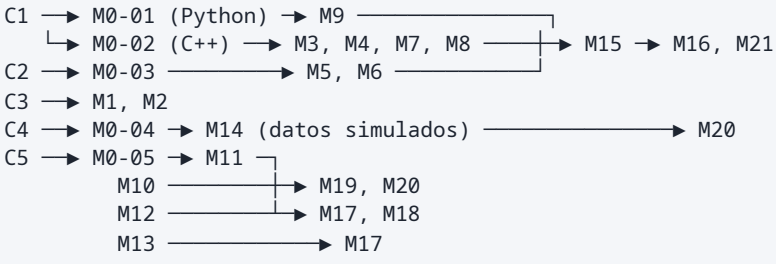
Reglas para cualquier adaptador:

1. **Nunca** se acredita por la redirección del navegador. Solo se acredita tras un webhook verificado **y** un `consultar()` que confirme `pagado` con el **mismo monto**.
2. El webhook simulado se firma así: `X-Firma: t=<unix>,v1=<hex HMAC-SHA256(PAGOS_SECRETO_WEBHOOK, "<t>." + cuerpo)>`. Se rechaza si `t` difiere más de 300 s de la hora del servidor.
3. Nunca se guardan datos de tarjetas bancarias: el pago ocurre en la `url_pago` de la pasarela.

10. Mapa de módulos, instancias y dependencias

Instancia 0: contratos en código; es corta y desbloquea al resto. Instancia 1: módulos independientes que se trabajan en paralelo contra los contratos, con simuladores. Instancia 2: integran módulos de la instancia 1; se pueden empezar antes con simuladores, pero se cierran después.

Módulo	Área	Inst.	Depende de	Issues
M0 Contratos en código	todas	0	—	6
M1 Tarjeta 215 → 424	tarjeta	1	C3	8
M2 Claves en el validador	firmware	1	C3	4
M3 Almacenamiento local v2	firmware	1	C1	6
M4 Sincronizador v2	firmware	1	C1, C2, M0-02	4
M5 Enlace 4G	firmware / hardware	1	C2	4
M6 Enlace WiFi	firmware	1	C2	2
M7 BLE para el conductor (ESP32)	firmware	1	C1, C2	4
M8 ESP-NOW entre buses	firmware	1	C1, C2	4
M9 Sincronización v2 en el servidor + MQTT	servidor	1	C1, M0-01	8
M10 Libro mayor	servidor	1	—	6
M11 Pagos y recargas	servidor	1	C4, C5	8
M12 Identidad del cliente	servidor	1	C4	8
M13 Notificaciones	servidor	1	—	4
M14 Portal del cliente (interfaz)	portal	1	C4, M0-04	9
M15 Gestor de enlaces	firmware	2	M4, M5, M6, M7	4
M16 Relé web del conductor	portal / servidor	2	M7, M9	4
M17 Robo y reposición de punta a punta	todas	2	M9, M12, M13, M14	3
M18 Emisión estilo bancario	servidor / PC	2	M12	3
M19 Panel de operadores, ampliación	servidor	2	M9, M10, M11, M12	4
M20 Integración del portal	portal	2	M10–M14, M1-08	3
M21 Pruebas de robustez	todas	2	todo	2



Camino crítico: M0-01/M0-02 (bibliotecas C1 con vectores) desbloquean todo el firmware y el servidor de sincronización. Conviene cerrarlas primero. M10, M12 y M13 no dependen de nada y pueden empezar el primer día.

11. Instancia 0: M0 contratos en código

> **Épica M0. Contratos en código.** Pasar C1–C5 a código que los demás importan. Sin estas piezas cada módulo inventaría su versión de los bytes.

M0-01 Biblioteca C1 en Python y vectores de prueba instancia 0 servidor

Depende de: C1 · **Archivos:** `servidor_api/transporte/sync_v2/__init__.py`, `paquetes.py`, `transporte/management/commands/generar_vectores_c1.py`, `docs/contratos/vectores_c1.json`, `requirements.txt` (+ `cryptography`)

Qué hacer

1. Crear las `dataclass` `Cabecera`, `Evento`, `Hola`, `Acuse`, `Listas` (con `EntradaLista`) y `Tarifas`, con los campos y tamaños exactos de C1 (5.1–5.6).
2. Funciones `empaquetar(cabecera, cuerpo, clave_hmac=None, clave_ecdsa=None) -> bytes`, `desempaquetar(datos, clave_hmac_de: Callable[[int], bytes], clave_publica) -> Paquete` (lanza `PaqueteInvalido` con el motivo) y `separar(flujo: bytes) -> list[bytes]` para el cuerpo HTTP concatenado.
3. Firmas: HMAC con `hmac` / `hashlib` (truncado a 16 B) y ECDSA P-256 con `cryptography`, convirtiendo DER ⇄ `r||s` de 64 B.
4. Comando `generar_vectores_c1`: con claves de prueba fijas (HMAC = 32 × `0x11`, ECDSA = un escalar fijo documentado en el JSON), genera ≥ 8 casos (EVENTOS con 1 y 30 eventos, HOLA, ACUSE, LISTAS normal, urgente y foto completa, TARIFAS) más 4 inválidos (magia, versión, largo, firma), con entradas en claro y salida en hex.

Criterios de aceptación

- ☐ Ida y vuelta (`desempaquetar(empaquetar(x)) == x`) en todos los tipos.
- ☐ Los tamaños coinciden byte a byte con C1 (test por cada tipo).
- ☐ Los 4 vectores inválidos lanzan `PaqueteInvalido` con el motivo correcto.
- ☐ `vectores_c1.json` está en el repositorio.

M0-02 Biblioteca C1 en C++ para la ESP32 instancia 0 firmware

Depende de: M0-01 (vectores) · **Archivos:** `pantalla_esp32/lib/ProtocoloSP/ProtocoloSP.h/.cpp`, `Fragmentos.h/.cpp`, `pantalla_esp32/test/test_protocolo/`, `herramientas/vectores_a_h.py`

Qué hacer

1. Structs con los mismos campos de C1 y funciones `size_t empaquetar(...)` / `bool desempaquetar(...)` sobre búferes estáticos (sin `String` ni memoria dinámica en el camino del cobro).
2. HMAC con `mbedtls_md_hmac` (SHA-256). Verificación ECDSA con micro-ecc (`lib/micro-ecc/`, ~67 ms frente a ~290 ms de `mbedtls_ecdsa_verify`) y la clave pública de `clave_servidor.h` (punto sin comprimir de 65 B).
3. `Fragmentos`: dividir un paquete en tramas "EN" FRAGMENTO de 220 B y reensamblarlas con un límite de 3 s (C1 5.10). Lo comparten M7 y M8.
4. Script `vectores_a_h.py`: convierte `vectores_c1.json` en `test/test_protocolo/vectores.h`.

Criterios de aceptación

- ☐ `pio test -e validador -f test_protocolo` pasa en la placa: produce exactamente los bytes de los vectores válidos y rechaza los inválidos.
- ☐ Verificar una firma ECDSA tarda < 100 ms en la ESP32 con micro-ecc (medido e impreso en el test). Solo ocurre al sincronizar, nunca al cobrar, y debe ir en una tarea aparte de `loop()`.
- ☐ Fragmentar y reensamblar paquetes de 1, 220, 221 y 2048 B funciona; un fragmento perdido descarta el mensaje.

M0-03 Interfaces de enlace (C2) e implementaciones falsas instancia 0 firmware

Depende de: C2 · **Archivos:** `pantalla_esp32/lib/Enlaces/Enlace.h`, `CanalPaquetes.h`, `Vecinos.h`, `EnlaceFalso.h`, `CanalFalso.h`

Qué hacer

1. Copiar las interfaces de C2 tal cual, con comentarios en español.
2. `CanalFalso`: guarda en una cola lo enviado y entrega lo que el test le inyecta; tiene `ponerListo(bool)`. `EnlaceFalso`: estado controlable por el test.

Criterios de aceptación

- ☐ Compila en el entorno `validador` y un test de ejemplo usa `CanalFalso`.

M0-04 OpenAPI de C4 y cliente JS con modo simulado instancia 0 portal

Depende de: C4 · **Archivos:** `docs/contratos/api-cliente-v1.yaml`, `servidor_api/portal/static/portal/js/api.js`, `api-simulada.js`

Qué hacer

1. Escribir el OpenAPI 3.1 con todas las rutas, esquemas, errores y ejemplos de la sección 8.
2. `api.js`: una función por ruta (`ingresar(documento, contrasena)`, `resumen()`, `movimientos(filtros)`, `bloquearTarjeta(id, motivo, contrasena)`, `crearOrden(monto)` …) que hace `fetch` con la cookie y la CSRF y convierte los errores en `ErrorApi{codigo, mensaje, campos}`.
3. `api-simulada.js`: las mismas funciones con datos falsos coherentes (cliente "Ana Quispe", saldo S/ 12.40, 45 movimientos, una tarjeta activa) y retardos de 300 ms. Se activa con `?simulado=1` o `localStorage.simulado = "1"`.

Criterios de aceptación

- ☐ El YAML pasa un validador de OpenAPI (por ejemplo `npx @redocly/cli lint`).
- ☐ Las firmas de `api.js` y `api-simulada.js` son idénticas.
- ☐ El modo simulado puede producir cada código de error (con `?error=<codigo>`).

M0-05 Interfaz de pasarela (C5) y pasarela falsa para tests instancia 0 servidor

Depende de: C5 · **Archivos:** `servidor_api/pagos/__init__.py`, `apps.py`, `pasarelas/base.py`, `pasarelas/falsa.py`, `pasarelas/__init__.py` (`obtener_pasarela()`)

Qué hacer

1. Crear la app `pagos` (registrada en `INSTALLED_APPS`, todavía sin modelos) con `base.py` exactamente como C5.
2. `PasarelaFalsa`, en memoria, para tests: permite fijar el resultado de `consultar()` y generar webhooks firmados.
3. `obtener_pasarela()` lee `settings.PAGOS_PASARELA`.

Criterios de aceptación

- ☐ Hay un test que usa `PasarelaFalsa` en cada método de la interfaz.

M0-06 Claves del protocolo: ECDSA del servidor y HMAC por validador instancia 0 servidor

Depende de: M0-01 · **Archivos:** `transporte/management/commands/generar_clave_listas.py`, `clave_validador.py`, `transporte/models.py` (campo `Validador.clave_hmac`), `.env.example`, `pantalla_esp32/src/validador/clave_servidor.h`

Qué hacer

1. `generar_clave_listas`: crea un par P-256, imprime la privada en PEM para `LISTAS_CLAVE_PRIVADA` en `.env` y escribe `clave_servidor.h` con la pública. Si ya existe una clave, se niega a sobrescribirla salvo con `--forzar`.
2. Campo `Validador.clave_hmac` (64 hex, generado con `secrets.token_bytes(32)` al crear el validador) y una migración que lo rellene en los existentes.
3. `clave_validador <numero> [--rotar]`: muestra o rota la clave para cargarla en la ESP32 (M2-02).

Criterios de aceptación

- ☐ Tests: los paquetes firmados con la clave del validador 101 no verifican con la del 102.
- ☐ `.env.example` documenta `LISTAS_CLAVE_PRIVADA` sin ningún valor real.

12. Instancia 1: módulos independientes

M1. Tarjeta NTAG215 → NTAG 424

> **Épica M1.** Que el sistema funcione hoy con la NTAG215 y mañana con la NTAG 424 sin tocar nada fuera de `transporte/tarjeta/` y `lib/TarjetaTransporte/`. Mientras no lleguen las 424, todo se prueba con un **simulador**.

M1-01 Interfaz `TarjetaFisica` y adaptar NTAG215

instancia 1 · tarjeta

Depende de: C3 · **Archivos:** `transporte/tarjeta/base.py` (nuevo), `ntag215.py`, programas de `transporte/` que usan `ntag215`

Qué hacer

1. Crear `base.py` con el `Protocol` de C3 y la función `abrir_tarjeta(conexion) -> TarjetaFisica`, que detecta el tipo (con GET_VERSION en Ultralight o con la respuesta ISO-DEP en la 424).
2. Hacer que `ntag215.py` implemente el protocolo sin cambiar su comportamiento.
3. Cambiar `emitir_tarjeta.py`, `validador.py`, `recargar.py`, `leer_tarjeta.py`, `bloquear_tarjeta.py` y `restablecer_tarjeta.py` para que usen `abrir_tarjeta()`.

Criterios de aceptación

- ☐ Emitir, cobrar, recargar en punto y bloquear con una NTAG215 real funciona igual que antes (prueba manual documentada en el PR).
- ☐ Ningún programa importa `ntag215` directamente.

M1-02 Especificación de personalización de la NTAG 424

instancia 1 · tarjeta · docs

Depende de: C3 · **Archivos:** `docs/ntag424-personalizacion.md`

Qué hacer

1. Con la hoja de datos NT4H2421Gx y las notas AN12196 y AN10922 de NXP, documentar **byte a byte**: los comandos `ChangeKey` (K0–K4), el `ChangeFileSettings` del archivo 02 (SDM activado, offsets de PICCData, SDMMAC y SDMMACInput para la URL de C3) y el del archivo 03 (lectura K1, escritura K2, cambio K0, CommMode Full), el contenido NDEF del archivo 02 y el orden de los pasos.
2. Incluir la plantilla de la URL con los offsets calculados y un ejemplo completo de las APDU en hex.
3. Confirmar en la hoja de datos el comportamiento ante **escrituras interrumpidas** (tearing) de `WriteData` y anotar la conclusión. El doble registro se mantiene en cualquier caso.

Criterios de aceptación

- ☐ Otra persona puede escribir `personalizar_424.py` (M1-06) solo con este documento.
- ☐ Las referencias citan la sección de la hoja de datos de cada valor.

M1-03 Simulador de NTAG 424 DNA en Python instancia 1 tarjeta

Depende de: C3 · **Archivos:** `transporte/tarjeta/simulador424.py`, `transporte/tests/test_simulador424.py`, `transporte/requirements.txt` (+ `pycryptodome`)

Qué hacer

1. Clase `Simulador424` con `transmitir(apdu: bytes) -> bytes` que implementa: ISOSelectFile, `GetVersion`, `AuthenticateEV2First` (y su segunda parte), `ReadData` y `WriteData` (CommMode Plain/MAC/Full), `GetFileSettings`, `ChangeFileSettings`, `ChangeKey`, `GetFileCounters` e ISOReadBinary del archivo 02 con SUN.
2. Estado: 5 claves (de fábrica en ceros), 3 archivos, contador SDM y contadores de comando de la sesión segura. Opción `cortar_en_byte=N` para simular el retiro de la tarjeta a mitad de una escritura.
3. Validar la mensajería segura con los **vectores de AN12196**.

Criterios de aceptación

- ☐ Los vectores de autenticación, derivación de claves de sesión, MAC y cifrado de AN12196 pasan.
- ☐ Leer el archivo 03 sin autenticar, o con K1 si se pide K2, devuelve el error que indica la hoja de datos.

M1-04 Controlador NTAG 424 en Python (ACR122U) instancia 1 tarjeta

Depende de: M1-01, M1-02, M1-03, M1-05 · **Archivos:** `transporte/tarjeta/ntag424.py`, `transporte/tests/test_ntag424.py`

Qué hacer

1. Implementar `TarjetaFisica` para la 424 enviando APDU por un `transmitir()` inyectable (pyscard con el ACR122U en producción y `Simulador424` en los tests).
2. `autenticar()` con K1 para leer y K2 para escribir (claves diversificadas con M1-05). `contador()` = `SDMReadCtr`.

Criterios de aceptación

- ☐ Contra el simulador: se emite, se cobra 10 veces, se corta una escritura y la lectura sigue devolviendo el registro anterior válido.
- ☐ Con una 424 real (cuando llegue): la misma prueba manual.

M1-05 Diversificación de claves AN10922 instancia 1 tarjeta

Depende de: C3 · **Archivos:** `transporte/tarjeta/claves.py`, test, `docs/contratos/vectores_an10922.json`

Qué hacer

1. `diversificar_aes128(maestra: bytes, uid: bytes, sistema=b"SubePe") -> bytes` según AN10922 (entrada `0x01 || UID || sistema`, relleno y CMAC).
2. Comprobar con el ejemplo de AN10922 y generar vectores propios para el firmware (M2-04).

Criterios de aceptación

- ☐ El vector de AN10922 pasa y los vectores propios están en el repositorio.

M1-06 Script `personalizar_424.py` instancia 1 tarjeta

Depende de: M1-02, M1-04 · **Archivos:** `transporte/personalizar_424.py`

Qué hacer

1. Pasa una 424 de fábrica (claves en cero) a la configuración de M1-02, en el orden documentado y verificando cada paso.
2. `--restablecer`: vuelve a la configuración de fábrica usando K0 (para reutilizar tarjetas de prueba).
3. `--simulador`: ejecuta todo contra `Simulador424`.

Criterios de aceptación

- ☐ Personalizar y restablecer en el simulador deja exactamente los ajustes de archivo esperados (comparados con `GetFileSettings`).
- ☐ Si se interrumpe a la mitad, volver a ejecutarlo termina el trabajo (es idempotente).

M1-07 Controlador NTAG 424 en el firmware instancia 1 firmware

Depende de: M1-02, M1-05 · **Archivos:** `pantalla_esp32/lib/TarjetaTransporte/Tarjeta424.h/.cpp`, `lib/LectorNFC/LectorPN532.cpp` (intercambio ISO-DEP si hace falta), `test/test_tarjeta424/`

Qué hacer

1. Mismo protocolo de C3 en C++ sobre `LectorNFC` (PN532 InDataExchange), con AES y CMAC de mbedTLS.
2. Tests con **trazas de APDU grabadas** del simulador (M1-03): para cada operación, las APDU esperadas y las respuestas.

Criterios de aceptación

- ☐ Los tests de trazas pasan en la placa.
- ☐ Con una 424 real: el cobro completo (autenticar, leer, escribir y verificar) tarda < 400 ms, medido.

M1-08 Verificación SUN en el servidor (vista pública) instancia 1 servidor

Depende de: M1-02, M1-05 · **Archivos:** `servidor_api/transporte/sun.py`, ruta `/t` en el host `mi.subepe.app`, plantilla, campo `Tarjeta.contador_sdm`

Qué hacer

1. Descifrar PICCData con K3, obtener el UID y el contador, derivar K4(UID) y verificar el CMAC. Rechazar si el contador ≤ `contador_sdm` guardado (repetición); si es mayor, guardarlo.
2. Página pública: "Tarjeta SubePe auténtica · estado: activa/bloqueada" y un botón "Ver mi saldo" que lleva al inicio de sesión. **No muestra saldo ni nombre** sin sesión. La vista del dueño es M20-02.

Criterios de aceptación

- ☐ Tests con URL generadas por el simulador: válida, CMAC alterado, contador repetido y tarjeta inexistente.

M2. Claves en el validador

> **Épica M2.** Hoy la clave maestra está **en texto plano en la microSD** (`/config.txt`): cualquiera que saque la tarjeta SD la obtiene. Con la 424 el riesgo pasa a ser el validador, así que hay que endurecerlo.

M2-01 Mover la clave maestra de la microSD a la NVS instancia 1 firmware

Depende de: — · **Archivos:** `src/validador/almacen.cpp/.h`, `transporte/puente_nfc.py`

Qué hacer

1. Al arrancar, si `/config.txt` tiene `clave_maestra`: copiarla a la NVS (`Preferences`, espacio `"claves"`), comprobar la copia y **borrar la línea** del archivo.
2. `claveMaestra()` lee de la NVS. `puente_nfc.py --configurar` la envía por serie directo a la NVS.

Criterios de aceptación

- ☐ Después del primer arranque, `config.txt` ya no contiene la clave y el validador sigue cobrando.
- ☐ Sacar la microSD y leerla en un ordenador no revela ninguna clave.

M2-02 Clave HMAC del validador (K_val) en la NVS instancia 1 firmware

Depende de: M0-06 · **Archivos:** `almacen.cpp/.h`, `puente_nfc.py` (opción `--clave-validador HEX`)

Qué hacer

1. Guardar y leer K_val (32 B) en la NVS, con `Almacen::claveValidador()`.
2. El portal de configuración y la pantalla muestran "Clave de validador: falta / OK", nunca la clave.

Criterios de aceptación

- ☐ Un paquete firmado en la placa verifica en el servidor con `clave_validador <n>`.

M2-03 Investigar el cifrado de flash y Secure Boot v2 (placa de pruebas) instancia 1 firmware docs

Depende de: — · **Archivos:** `docs/informe-cifrado-flash.md`

> **Irreversible:** quemar eFuses no tiene vuelta atrás y puede dejar la placa inutilizable. Solo se hace en una placa **dedicada a pruebas**, nunca en el validador principal.

Qué hacer

1. Comprobar si con `framework = arduino` se puede activar el cifrado de flash y de NVS, o si hace falta `framework = arduino, espidf` para personalizar `sdkconfig`.
2. Activarlo en modo *development* en la placa de pruebas y documentar los comandos, el efecto en el tiempo de arranque y cómo se cargan las actualizaciones.
3. Recomendar si se adopta, con sus pasos.

Criterios de aceptación

- ☐ Informe con el procedimiento reproducible y la recomendación.
- ☐ Evidencia: el volcado de la flash con `esptool read_flash` no muestra la clave.

M2-04 Diversificación AN10922 en el firmware instancia 1 firmware

Depende de: M1-05 (vectores) · **Archivos:** `lib/TarjetaTransporte/Claves424.h/.cpp`, `test/test_claves424/`

Qué hacer

1. La misma función que M1-05, con mbedTLS (`mbedtls_cipher_cmac`).

Criterios de aceptación

- ☐ Los vectores de `vectores_an10922.json` pasan en la placa.

M3. Almacenamiento local v2 (microSD)

> **Épica M3.** Pasar de archivos de texto JSON a archivos binarios con el formato C1, que resisten cortes de energía y escalan a listas grandes. La interfaz pública de `Almacen` que usa el cobro se mantiene.

M3-01 Cola binaria de eventos instancia 1 firmware

Depende de: M0-02 · **Archivos:** `src/validador/almacen.cpp/.h`

Qué hacer

1. `/eventos.bin`: registros de 64 B (evento C1 5.3), solo se agregan al final y se hace `flush()` antes de mostrar el resultado del cobro.
2. Al arrancar: si el tamaño no es múltiplo de 64, se trunca el último registro incompleto y se registra la incidencia.
3. Migración: si existe `/eventos.txt`, se convierten sus eventos pendientes y se renombra a `/eventos.txt.migrado`.
4. Compactación: cuando más de la mitad de los registros ya tienen acuse, se reescribe el archivo (temporal + `rename`).

Criterios de aceptación

- ☐ Prueba de corte: cortar la energía 20 veces durante cobros seguidos no pierde ningún evento confirmado en pantalla.
- ☐ La migración desde un `eventos.txt` real conserva todos los pendientes.

M3-02 Estado persistente con escritura atómica instancia 1 firmware

Depende de: — · **Archivos:** `almacen.cpp/.h`

Qué hacer

1. `/estado.bin`: `siguiente_id`, `hasta_acusado`, `version_listas`, `version_tarifas`, `secuencia_paquetes` y un CRC32. Se escribe en `/estado.tmp` y luego se renombra.
2. Si el CRC falla, se usa `/estado.bak` (la versión anterior).

Criterios de aceptación

- ☐ `secuencia_paquetes` y `siguiente_id` nunca se repiten tras un corte de energía (50 cortes de prueba).

M3-03 Lista negra ordenada con búsqueda binaria y cambios instancia 1 firmware

Depende de: M0-02 · **Archivos:** `almacen.cpp/.h`

Qué hacer

1. `/lista_negra.bin`: uint32 ordenados. `bloqueada(id)` hace una búsqueda binaria en el archivo, sin cargarlo en RAM.
2. `aplicarCambios(entradas)`: mezcla las altas y bajas en un `.tmp` y lo renombra. Aplicar un mismo cambio dos veces no cambia el resultado.
3. `reiniciarListas()` para `version_desde = 0`.
4. `bloqueoUrgente(id)`: lista pequeña en RAM más `/urgentes.bin`, que se consultan junto con la principal.

Criterios de aceptación

- ☐ Con 50 000 ids, una consulta tarda < 20 ms (medido en la placa).
- ☐ Aplicar 120 cambios tarda < 2 s y no bloquea el cobro más de 50 ms seguidos (se hace por partes en `loop()`).

M3-04 Recargas pendientes y tarifas en binario instancia 1 firmware

Depende de: M0-02 · **Archivos:** `almacen.cpp/.h`

Qué hacer

1. `/recargas.bin`: registros (cuenta, seq, monto) ordenados por cuenta y seq; se aplican los cambios de tipo 2 de LISTAS.
2. `/tarifas.bin`: el cuerpo TARIFAS tal cual. `recargasPendientes()` y `tarifa()` mantienen su firma actual.

Criterios de aceptación

- ☐ El cobro con recarga pendiente funciona igual que hoy (prueba con una tarjeta real).

M3-05 Paquetes ajenos y paquetes firmados reenviables instancia 1 firmware

Depende de: M0-02 · **Archivos:** `almacen.cpp/.h`

Qué hacer

1. `/ajenos/<origen>-<secuencia>.bin`: paquetes de otros buses por subir, hasta 500 (si se llena, se borra el más antiguo). Se borran cuando llega el ACUSE del origen que los cubre.
2. `/firmados/<desde>-<hasta>.bin`: los últimos 50 paquetes LISTAS recibidos del servidor, para dárselos a otros buses (M8).

Criterios de aceptación

- ☐ Hay un test con `CanalFalso`: guardar, listar, borrar por acuse y respetar el límite de 500.

M3-06 **Contador de consumo de datos por canal** instancia 1 firmware

Depende de: — · **Archivos:** `almacen.cpp/.h`

Qué hacer

1. `/consumo.bin`: por día (AAAAMMDD) y canal (wifi, 4g, ble, espnow), bytes de subida y de bajada. Se guarda cada 5 min y se conservan 31 días.
2. Expone `bytesHoy(tx, rx)` para el HOLA.

Criterios de aceptación

- ☐ Los valores coinciden con los contadores de los enlaces en una prueba de 1 hora.

M4. Sincronizador v2 (firmware)

Épica M4. Reemplaza la lógica de `sincronizador.cpp` (JSON por HTTP) por el intercambio de paquetes C1 sobre un `CanalPaquetes`. No sabe qué canal usa.

M4-01 **Intercambio HOLA / EVENTOS / respuestas** instancia 1 firmware

Depende de: M0-02, M0-03 · **Archivos:** `src/validador/sincronizador_v2.cpp/.h`

Qué hacer

1. `SincronizadorV2::atender(CanalPaquetes&)`. Al pasar el canal a `listo()` envía HOLA; después envía los eventos pendientes en paquetes de hasta 30, como máximo 3 paquetes sin acuse a la vez; un viaje nuevo se envía en ≤ 1 s si el canal está listo; HOLA cada 5 min.
2. También sube los paquetes de `/ajenos/` tal cual, sin volver a firmarlos.
3. Procesa ACUSE (marcar subidos y borrar ajenos), LISTAS y TARIFAS (M4-02).

Criterios de aceptación

- ☐ Test con `CanalFalso`: 100 eventos salen en 4 paquetes; un acuse hasta 60 deja 40 pendientes; se reenvía tras 30 s sin acuse.

M4-02 **Aplicación de LISTAS y TARIFAS según C1** instancia 1 firmware

Depende de: M4-01, M3-03, M3-04 · **Archivos:** `sincronizador_v2.cpp`

Qué hacer

1. Implementar las 5 reglas de C1 5.5 (contigüidad, `desde = 0`, foto completa encadenada, urgentes y firma inválida → incidencia).
2. Guardar cada LISTAS válido en `/firmados/`.

Criterios de aceptación

- ☐ Hay un test por regla con los paquetes de `vectores_c1.json` y paquetes alterados.

M4-03 **Hora del servidor desde el ACUSE** instancia 1 firmware

Depende de: M4-01 · **Archivos:** `sincronizador_v2.cpp`, `main.cpp`

Qué hacer

1. Tomar `hora_servidor` de cada ACUSE con firma válida y exponer `horaServidor()` y `millisHora()` como hoy. Si se recibe por un canal reenviado (BLE o ESP-NOW), descontar el retraso estimado (+2 s).

Criterios de aceptación

- ☐ La hora del validador queda a ±3 s de la del servidor tras sincronizar por cualquier canal.

M4-04 Pruebas de punta a punta del sincronizador con un servidor local instancia 1 firmware

Depende de: M4-01, M9-04 (o un servidor falso) · **Archivos:** `herramientas/servidor_c1_falso.py`

Qué hacer

1. Script Python que habla C1 por HTTP (`/api/v2/paquetes/`) con M0-01, para probar el firmware sin el servidor real. Imprime cada paquete decodificado.

Criterios de aceptación

- ☐ Se documenta en `pantalla_esp32/README.md` cómo usarlo.

M5. Enlace 4G

> **Épica M5.** Conexión a internet propia del validador con un módulo celular Cat-1 y una SIM del sistema, para no depender del conductor.

M5-01 Compra, montaje y cableado del módulo instancia 1 hardware

Depende de: — · **Archivos:** `pantalla_esp32/README.md` (sección "Módulo 4G")

Qué hacer

1. Comprar el módulo de la sección 14 (variante **SA**, para las bandas de Sudamérica) y una SIM prepago de datos.
2. Alimentación aparte: **5 V y 2 A** como mínimo (picos de ~2 A al transmitir). **No** se alimenta desde el pin 3V3 de la ESP32. Unir las tierras.
3. UART2 de la ESP32: **TX = GPIO25 → RXD del módulo**; **RX = GPIO34 ← TXD del módulo** (GPIO34 es solo entrada, sirve para RX); **PWRKEY = GPIO4** a través de un transistor NPN si la placa no lo trae. Son pines libres en el validador actual.
4. Comprobar el **nivel lógico** de la placa del módulo: el chip trabaja a 1,8 V; usar una placa con adaptador de niveles a 3,3 V o poner uno.

Criterios de aceptación

- ☐ README con un diagrama de conexiones y una foto del montaje.
- ☐ `AT` → `OK` desde un programa de eco por serie.

M5-02 Programa de prueba `prueba4g` instancia 1 firmware

Depende de: M5-01 · **Archivos:** `src/prueba_4g.cpp`, `platformio.ini` (entorno `prueba4g`)

Qué hacer

1. Con TinyGSM 0.12.x (perfil del módem SIM7600 o A7672X, según el módulo comprado; verificar que funcione): encender, leer el estado de la SIM, registrarse, mostrar la señal, configurar el APN (`config.txt` : `apn=`), abrir TCP a `subepe.app:443` y negociar **TLS en la ESP32** con ESP_SSLClient y las raíces de `raices_tls.h` .
2. Imprimir el tiempo de cada paso.

Criterios de aceptación

- ☐ Un `GET /` por TLS sobre 4G devuelve la redirección del servidor.
- ☐ Tiempos reportados en el PR (registro en la red, TCP, TLS).

M5-03 `Enlace4G` **(C2)** instancia 1 firmware

Depende de: M5-02, M0-03 · **Archivos:** `lib/Enlaces/Enlace4G.h/.cpp`

Qué hacer

1. Máquina de estados que no bloquea: Apagado → encendiendo (PWRKEY) → esperando SIM → registrando → datos → Listo. Cualquier fallo pasa a Error, con reintento en 30 s, 60 s y luego cada 120 s. Ninguna llamada bloquea más de 50 ms (comandos AT asíncronos).
2. `abrir()` devuelve el cliente TLS; `bytesTx()` / `bytesRx()` cuentan los bytes del flujo TLS.
3. `apagar()` desconecta y apaga el módem (AT+CPOF o PWRKEY).

Criterios de aceptación

- ☐ Prueba de 2 h con la antena desconectada y reconectada 5 veces: se recupera solo y el cobro nunca se bloquea (medir el tiempo máximo de `loop()`).

M5-04 **Medición de consumo real de datos (24 h)** instancia 1 firmware docs

Depende de: M5-03, M15-02 (o MQTT de prueba) · **Archivos:** `docs/informe-consumo-datos.md`

Qué hacer

1. 24 h conectado por MQTT con 1000 viajes simulados (un evento cada ~86 s). Registrar los bytes por hora y los que informa el operador.

Criterios de aceptación

- ☐ Informe con el consumo por día y la proyección mensual. El objetivo es < 1 MB/día; si se supera, se explica por qué.

M6. Enlace WiFi

> **Épica M6.** Separar el WiFi que ya funciona (router o hotspot del celular) de `sincronizador.cpp` y ponerlo detrás de la interfaz `Enlace`.

M6-01 `EnlaceWifi` **(C2)** instancia 1 firmware

Depende de: M0-03 · **Archivos:** `lib/Enlaces/EnlaceWifi.h/.cpp`, `src/validador/sincronizador.cpp`, `portal.cpp`

Qué hacer

1. Mover la conexión a las redes guardadas (en orden, sin bloquear), el *keep-alive* TLS y mDNS a `EnlaceWifi`. El portal de configuración no cambia.
2. `abrir()` admite `https://` (con `raices_tls.h`) y, en pruebas locales, `http://IP:puerto`.

Criterios de aceptación

- ☐ El validador sincroniza por WiFi igual que antes (v1) mientras M15 no esté terminado.

M6-02 **Canal de radio y convivencia con ESP-NOW** instancia 1 firmware

Depende de: M6-01 · **Archivos:** `EnlaceWifi.cpp`

Qué hacer

1. Exponer `canal()`, el canal del AP conectado. Avisar por una función de retorno cuando cambie, para que `Vecinos::pausar()` se active si es distinto de 1.

Criterios de aceptación

- ☐ Con un AP en el canal 1, ESP-NOW y WiFi funcionan a la vez (prueba con M8-01).

M7. BLE para el celular del conductor (lado ESP32)

> **Épica M7.** Servicio Bluetooth LE por el que una página web del conductor (M16) se lleva la cola del validador y le trae las listas.

M7-01 NimBLE, partición y memoria instancia 1 firmware

Depende de: — · **Archivos:** `platformio.ini`, `docs/informe-memoria-validador.md`

Qué hacer

1. Agregar `h2zero/NimBLE-Arduino@^1.4` (compatible con el núcleo Arduino 2.x) y la partición `huge_app.csv` si el binario no cabe.
2. Medir la RAM libre y el bloque mayor con WiFi + TLS + BLE activos a la vez.

Criterios de aceptación

- ☐ Con todo activo quedan ≥ 40 KB de memoria dinámica libre (reportado en el informe).

M7-02 Servicio GATT del validador instancia 1 firmware

Depende de: M7-01, M0-02 · **Archivos:** `lib/Enlaces/CanalBle.h/.cpp`

Qué hacer

1. Anunciar el nombre `SubePe-V<numero>` con el UUID de servicio de C1 5.10 y las 4 características.
2. Control 1 (empezar): enviar por notify las tramas FRAGMENTO de HOLA, los pendientes y los ajenos. Entrada: reensamblar con `Fragmentos` y entregar con `recibir()`.
3. Pedir MTU 247. Un solo celular conectado a la vez.

Criterios de aceptación

- ☐ Con la app nRF Connect: se lee **info** y se reciben notificaciones de **salida** tras escribir 1 en **control**.

M7-03 `CanalBle` como `CanalPaquetes` e indicación en el LCD instancia 1 firmware

Depende de: M7-02 · **Archivos:** `CanalBle.cpp`, `lib/PantallaUI/`

Qué hacer

1. `listo()` es verdadero cuando hay un celular conectado que escribió control = 1. Mostrar "Conductor conectado" en la barra de estado del LCD sin tapar el resultado del cobro.

Criterios de aceptación

- ☐ Un cobro durante una sesión BLE sigue tardando < 400 ms.

M7-04 Herramienta de PC `rele_ble.py` instancia 1 firmware

Depende de: M7-02, M0-01 · **Archivos:** `transporte/rele_ble.py` (librería `bleak`)

Qué hacer

1. Hace de "conductor" desde el ordenador: se conecta, se lleva la cola, la sube a `/api/v2/paquetes/` (o a `servidor_c1_falso.py`) y devuelve las respuestas. Sirve para probar M7 y M9 sin la página web.

Criterios de aceptación

- ☐ Ciclo completo: 50 eventos suben y el validador recibe el acuse y las listas.

M8. ESP-NOW entre buses

> **Épica M8.** Los buses que se cruzan o coinciden en el terminal se pasan listas (bloqueos) y se llevan los paquetes del otro hacia el servidor. Es gratis y no necesita infraestructura.

M8-01 Balizas y tabla de vecinos instancia 1 firmware

Depende de: M0-02, M0-03 · **Archivos:** `lib/Enlaces/VecinosEspNow.h/.cpp`

Qué hacer

1. Inicializar ESP-NOW en el canal 1 y difundir una BALIZA (C1 5.10) cada 2 s ± 500 ms.
2. Tabla de hasta 10 vecinos (MAC, validador, versiones, pendientes, flags, último visto); se olvidan tras 10 s sin balizas.

Criterios de aceptación

- ☐ Con 2 placas, cada una ve a la otra en < 3 s (registro por serie).

M8-02 Transferencia fiable de mensajes instancia 1 firmware

Depende de: M8-01 · **Archivos:** `VecinosEspNow.cpp`

Qué hacer

1. Enviar paquetes C1 en tramas FRAGMENTO por unicast. Usar el estado de entrega de la función de retorno de ESP-NOW: 3 reintentos por fragmento y, si fallan, se aborta el mensaje.
2. Como máximo 20 mensajes por encuentro y por sentido, para no acaparar la radio.

Criterios de aceptación

- ☐ Transferir 20 paquetes de 2 KB entre 2 placas a 10 m funciona sin pérdidas en 10 repeticiones.

M8-03 Reglas de intercambio entre buses instancia 1 firmware

Depende de: M8-02, M3-05 · **Archivos:** `VecinosEspNow.cpp`

Qué hacer

1. Si un vecino tiene `version_listas` mayor: mandarle un PEDIDO(1, mi versión). El vecino responde con sus `/firmados/` encadenados desde esa versión; se aplican como si vinieran del servidor (la firma se verifica igual).
2. Si el vecino tiene canal con el servidor (bit0), o si llevo más de 10 min sin canal y el vecino acepta ajenos (bit1): entregarle hasta 10 paquetes EVENTOS propios. El vecino los guarda en `/ajenos/`.
3. Al recibir un PEDIDO(3): entregar los ACUSE firmados que tenga para ese validador.

Criterios de aceptación

- ☐ Un paquete LISTAS con la firma alterada que llega por ESP-NOW se rechaza.

M8-04 Escenario de 2–3 buses en banco de pruebas instancia 1 firmware docs

Depende de: M8-03, M4-01 · **Archivos:** `docs/informe-espnow.md`

Qué hacer

1. Placa A con WiFi y placa B sin red. B cobra 30 viajes, A y B se "cruzan" (se encienden cerca 60 s) y los viajes de B llegan al servidor por A. Bloquear una tarjeta en el servidor: llega a A por MQTT y a B por ESP-NOW.

Criterios de aceptación

- ☐ Informe con tiempos: encuentro → viajes en el servidor; bloqueo → aplicado en B.

M9. Sincronización v2 en el servidor y MQTT

> **Épica M9.** El servidor acepta paquetes C1 por HTTP y MQTT, de cualquier validador y por cualquier intermediario, y produce las respuestas firmadas. Mosquitto y el puente van en el mismo Docker Compose.

M9-01 Modelos y migraciones instancia 1 servidor

Depende de: C1 · **Archivos:** `transporte/models.py`, `transporte/migrations/`

Qué hacer

1. `CambioLista`: `version` (BigAutoField, PK), `tipo` (1 bloqueo, 2 recarga), `op`, `a`, `b`, `c`, `urgente` (bool), `creado`, `difundido_en` (null).
2. `EventoRecibido`: (`validador`, `id_evento`) único, `recibido`, `via` (mqtt, http, ble, espnow).
3. `Validador`: `ultimo_evento_contiguo`, `version_listas`, `ultimo_hola` (JSON), `ultimo_enlace`, `mqtt_clave` (hash). `Tarifa.modificada` (`auto_now`).
4. Migración de datos: un `CambioLista` por cada tarjeta bloqueada o anulada y por cada recarga pendiente actual, para que la versión 1..N refleje el estado de hoy.

Criterios de aceptación

- ☐ Las migraciones se aplican sobre una copia de la base de producción (`restaurar.sh`) sin errores.

M9-02 Registrar los cambios de lista en los servicios instancia 1 servidor

Depende de: M9-01 · **Archivos:** `transporte/servicios.py`

Qué hacer

1. En la **misma transacción** de cada operación: `bloquear()` y `anular_por_uid()` → CambioLista(1, agregar, urgente = True); `crear_recarga()` → (2, pendiente); `entregar_recargas()` y `registrar_entrega_en_punto()` → (2, aplicada) por cada recarga; `confirmar_emision()` con tarjetas reemplazadas → (1, agregar).
2. Función `cambios_desde(version, limite=120)`.

Criterios de aceptación

- ☐ Hay un test por cada servicio que comprueba el CambioLista creado.
- ☐ Si la operación falla, no queda ningún cambio registrado (rollback).

M9-03 Servicio `procesar_paquete` instancia 1 servidor

Depende de: M0-01, M9-02 · **Archivos:** `transporte/sync_v2/servicio.py`

Qué hacer

1. `procesar(datos: bytes, via: str, portador: int | None) -> list[bytes]`: verifica la firma con la clave del **origen** (no del portador) y luego:
   EVENTOS → por cada evento, si no está en `EventoRecibido`, lo convierte al dict v1 y llama a `servicios.procesar_evento()`; actualiza `ultimo_evento_contiguo`; responde con un ACUSE para el origen.
   HOLA → guarda `ultimo_hola` y `ultima_sync`, y responde ACUSE + LISTAS (según `version_listas`, con foto completa si hay más de 2000 versiones de atraso) + TARIFAS si cambiaron.
2. Un paquete inválido no lanza excepción: se registra un `Alerta` "paquete inválido" con el motivo y se devuelve una lista vacía.

Criterios de aceptación

- ☐ Tests: idempotencia (el mismo paquete 3 veces = 1 viaje), paquete ajeno subido por otro validador, firma de otro validador, eventos desordenados (contigüidad), foto completa y tarifas.

M9-04 Endpoint HTTP `/api/v2/paquetes/` instancia 1 servidor

Depende de: M9-03 · **Archivos:** `transporte/sync_v2/vistas.py`, `transporte/urls.py`, `transporte/permisos.py` (`EsConductorOValidador`)

Qué hacer

1. Recibir octet-stream (máximo 64 KB), `separar()`, procesar cada paquete y devolver las respuestas concatenadas. `via` = "ble" si el usuario es conductor y "http" si es validador.
2. Limitar a 60 peticiones por minuto por usuario.

Criterios de aceptación

- ☐ Un usuario anónimo recibe 401, uno sin el grupo adecuado 403 y un cuerpo de más de 64 KB 400.

M9-05 Mosquitto en Docker Compose instancia 1 servidor

Depende de: C1 5.8 · **Archivos:** `servidor_api/mosquitto/mosquitto.conf`, `acl`, `docker-compose.yml`, `docker-compose.prod.yml`, `mosquitto/credenciales.sh`

Qué hacer

1. Servicio `mosquitto` (`eclipse-mosquitto:2`), `restart: unless-stopped`. Listener 1883 solo dentro de la red de Docker (para el puente). En producción, listener **8883 TLS** con el certificado de Let's Encrypt que ya obtiene Caddy (montar `caddy_data` en solo lectura y apuntar `certfile` / `keyfile` a `/data/caddy/certificates/acme-v02.api.letsencrypt.org-directory/<dominio>/`).
2. `allow_anonymous false`, `password_file` y el `acl_file` de C1. `persistence false`.
3. `credenciales.sh`: crea y actualiza el usuario `puente` y uno por validador activo (contraseña = `mqtt_clave`, generada por `manage.py clave_validador --mqtt`) con `mosquitto_passwd`, y recarga con `docker compose kill -s HUP mosquitto`.
4. Cron en la VM (junto al de `respaldar.sh`): `kill -s HUP` diario para que Mosquitto tome el certificado renovado.

Criterios de aceptación

- ☐ `mosquitto_sub` con el usuario 101 no puede leer `sp/v/102/baja` ni escribir en `sp/todos/urgente`.
- ☐ Conexión TLS desde fuera con `mosquitto_pub --capath /etc/ssl/certs`.
- ☐ La RAM de Mosquitto en reposo es < 15 MB (`docker stats`).

M9-06 Puente MQTT ↔ Django (`puente_mqtt`) instancia 1 servidor

Depende de: M9-03, M9-05 · **Archivos:** `transporte/management/commands/puente_mqtt.py`, `requirements.txt` (+ `paho-mqtt==2.1.*`), `docker-compose.yml` (servicio `puente`, misma imagen que `web`)

Qué hacer

1. Suscribirse a `sp/v/+/sube`. Por cada mensaje: `procesar(payload, "mqtt", portador=n)` y publicar cada respuesta en `sp/v/{n}/baja`, el portador, que reenvía a los demás buses por ESP-NOW lo que no es suyo.
2. Cada 2 s: los `CambioLista` urgentes con `difundido_en` nulo → paquete LISTAS urgente firmado → `sp/todos/urgente` → marcar `difundido_en`.
3. Reconexión con espera exponencial, cierre limpio con SIGTERM y conexión a la base de datos renovada en cada mensaje (`close_old_connections()`).

Criterios de aceptación

- ☐ Un bloqueo hecho en el panel llega como LISTAS urgente a un suscriptor de prueba en < 5 s.
- ☐ El puente se recupera solo tras reiniciar Mosquitto o PostgreSQL.

M9-07 Puerto 8883 en Oracle y guía de mudanza instancia 1 servidor docs

Depende de: M9-05 · **Archivos:** `README.md` (sección de despliegue)

Qué hacer

1. Abrir TCP 8883 en la Security List de la VCN y en iptables de la VM (además de 80 y 443). Agregar el paso a la guía de mudanza a la VM gratuita.

Criterios de aceptación

- ☐ `openssl s_client -connect subepe.app:8883` muestra el certificado de Let's Encrypt.

M9-08 Batería de tests de sincronización v2 instancia 1 servidor

Depende de: M9-03, M9-04 · **Archivos:** `transporte/tests/test_sync_v2.py`

Qué hacer

1. Reproducir en v2 los casos de fraude de `test_api.py` (clon, copia restaurada, tarjeta bloqueada usada sin conexión, saldo negativo) y añadir los de C1 (ajenos, urgentes, foto completa).

Criterios de aceptación

- ☐ Hay ≥ 15 tests nuevos y todos los tests v1 siguen pasando.

M10. Libro mayor (dinero real)

> **Épica M10.** Cada céntimo queda en asientos de doble partida inmutables. `Cuenta.saldo` se mantiene como caché, actualizada en la misma transacción, y se verifica todos los días contra el libro.

M10-01 Modelos contables inmutables

instancia 1 · servidor

Depende de: — · **Archivos:** nueva app `servidor_api/libro/` (`models.py`, `admin.py`, migraciones)

Qué hacer

1. `CuentaContable`: `codigo` único, `nombre`, `naturaleza` (activo, pasivo, ingreso), `cuenta_cliente` (FK a `transporte.Cuenta`, nula, única). Cuentas fijas creadas por migración: `CAJA_PASARELA`, `CAJA_VENTANILLA`, `INGRESOS_PASAJES`, `AJUSTES`. Una `SALDO_CLIENTE_<id>` (pasivo) por cuenta de cliente.
2. `Asiento`: `fecha`, `tipo` (recarga, viaje, emision, reverso, ajuste, apertura), `ref` única, `descripcion`, `creado_por` (nulo). `Partida`: `asiento`, `cuenta_contable`, `monto` (int; positivo = debe, negativo = haber).
3. Inmutables: `save()` sobre un objeto existente y `delete()` lanzan una excepción; en el admin son de solo lectura.

Criterios de aceptación

- ☐ Hay tests que intentan modificar y borrar y fallan.

M10-02 API del libro

instancia 1 · servidor

Depende de: M10-01 · **Archivos:** `libro/libro.py`

Qué hacer

1. `registrar(tipo, ref, partidas: list[tuple[str, int]], fecha=None, descripcion="") -> Asiento`: exige que la suma sea 0 y al menos 2 partidas; si la `ref` ya existe devuelve el asiento existente (idempotente) solo si las partidas son iguales, y si no lanza `ConflictoAsiento`.
2. `saldo_cliente(cuenta) -> int` (= −suma de las partidas de su SALDO_CLIENTE). `revertir(asiento, motivo)`: asiento espejo con `ref = "rev-" + ref`.
3. Convención documentada: recarga web de S/ 10 → CAJA_PASARELA +1000 / SALDO_CLIENTE −1000; viaje de S/ 1.30 → SALDO_CLIENTE +130 / INGRESOS_PASAJES −130.

Criterios de aceptación

- ☐ Tests de suma distinta de 0, idempotencia, conflicto y reverso.

M10-03 Integración con `registrar_movimiento`

instancia 1 · servidor

Depende de: M10-02 · **Archivos:** `transporte/servicios.py`

Qué hacer

1. Cada llamada a `registrar_movimiento()` registra su asiento en la misma transacción, con `ref` = `Movimiento.ref` o `"mov-<id>"`, y la contrapartida según el tipo: viaje → INGRESOS_PASAJES; recarga → según el origen (web → CAJA_PASARELA; punto o ventanilla → CAJA_VENTANILLA); emisión con saldo inicial → CAJA_VENTANILLA.

Criterios de aceptación

- ☐ Después de toda la suite de tests, `verificar_libro` (M10-05) da 0 diferencias.

M10-04 Migración de apertura

instancia 1 · servidor

Depende de: M10-01 · **Archivos:** migración de datos en `libro/migrations/`

Qué hacer

1. Por cada `Cuenta` con saldo distinto de 0: un asiento "apertura" (`ref = "apertura-<id>"`) contra AJUSTES por el saldo actual.

Criterios de aceptación

- ☐ Sobre una copia de producción: tras migrar, el libro cuadra con `Cuenta.saldo` en todas las cuentas.

M10-05 Comando `verificar_libro` y cron instancia 1 servidor

Depende de: M10-02 · **Archivos:** `libro/management/commands/verificar_libro.py`, `README.md`

Qué hacer

1. Comprueba que la suma global de las partidas sea 0 y que `Cuenta.saldo == saldo_cliente()` en cada cuenta. Cada diferencia crea un `Alerta` "descuadre contable" y el comando sale con código 1.
2. Cron diario a las 03:30 en la VM, después del respaldo.

Criterios de aceptación

- ☐ Hay un test que descuadra a propósito con un update SQL directo y se detecta.

M10-06 Documentación contable instancia 1 docs

Depende de: M10-02 · **Archivos:** `docs/libro-contable.md`

Qué hacer

1. Tabla de los asientos de cada operación (recarga web, recarga en punto, viaje, emisión, reembolso, ajuste) con un ejemplo numérico.

Criterios de aceptación

- ☐ Lo revisa otro miembro del equipo.

M11. Pagos y recargas

> **Épica M11.** Las recargas desde el portal tienen el flujo completo de un pago real. Solo la pasarela es simulada, detrás de C5.

M11-01 Modelos de órdenes y eventos de pasarela instancia 1 servidor

Depende de: M0-05 · **Archivos:** `pagos/models.py`, migraciones, `admin.py` (solo lectura)

Qué hacer

1. `OrdenRecarga`: `id` UUID, `cuenta` (FK), `usuario` (FK, nulo), `monto`, `estado` (creada, pendiente, pagada, acreditada, fallida, expirada, reembolsada), `pasarela`, `id_externo` (único con la pasarela), `clave_idempotencia` (única con el usuario), `url_pago`, `creada`, `expira` (+30 min), `pagada_en`, `acreditada_en`, `recarga` (FK a `transporte.Recarga`, nula), `datos_pasarela` (JSON).
2. `EventoPasarela`: `pasarela`, `id_evento` (único con la pasarela), `tipo`, `cuerpo` (bytes en crudo), `firma_valida`, `recibido`, `procesado_en`.
3. `HistorialOrden`: `orden`, `de`, `a`, `fecha`, `motivo` (auditoría de cada transición).

Criterios de aceptación

- ☐ Las restricciones de unicidad se prueban con tests de integridad.

M11-02 Máquina de estados, creación de órdenes y límites instancia 1 servidor

Depende de: M11-01 · **Archivos:** `pagos/servicios.py`

Qué hacer

1. `transicion(orden, a, motivo)` con la tabla permitida: creada → pendiente; pendiente → pagada | fallida | expirada; pagada → acreditada; acreditada → reembolsada. Cualquier otra lanza `TransicionInvalida`. Bloquea la fila con `select_for_update()` y escribe `HistorialOrden`.
2. `crear_orden(usuario, cuenta, monto, clave)`: 100 ≤ monto ≤ 20000; máximo S/ 500 al día por cuenta (suma de órdenes pagadas y acreditadas); máximo 3 pendientes; idempotencia según C4. Llama a `pasarela.crear_cobro()` y pasa a pendiente. Si la pasarela falla (`ErrorPasarela`), la orden queda creada y se puede reintentar con la misma clave.
3. `acreditar(orden)`: en una transacción, pagada → acreditada y `servicios.crear_recarga(cuenta, monto, origen="web")`, que registra la recarga pendiente de llegar a la tarjeta y su asiento.

Criterios de aceptación

- ☐ Tests de cada transición válida e inválida, límites e idempotencia.
- ☐ Prueba de concurrencia: 2 hilos que acreditan la misma orden → solo 1 recarga.

M11-03 Pasarela simulada instancia 1 servidor

Depende de: M0-05 · **Archivos:** `pagos/pasarelas/simulada.py`, `pagos/simulador/` (vistas, plantillas, modelo `CobroSimulado`)

Qué hacer

1. Implementar C5. `crear_cobro` crea un `CobroSimulado` y devuelve `url_pago = https://subepe.app/pagos/simulador/<id_externo>/`.
2. Página del simulador, claramente rotulada "PASARELA DE PRUEBA, NO ES UN PAGO REAL": muestra el monto y los botones "Aprobar" y "Rechazar". Al pulsar: actualiza el cobro, envía el webhook firmado (C5 regla 2) al endpoint de M11-04 con 1 s de retraso y redirige a `url_retorno`.
3. Opción del simulador para probar casos difíciles: "enviar webhook dos veces" y "no enviar webhook".

Criterios de aceptación

- ☐ Flujo completo en el navegador: orden → simulador → aprobar → acreditada.

M11-04 Endpoint de webhook con doble verificación instancia 1 servidor

Depende de: M11-02 · **Archivos:** `pagos/vistas.py`, `pagos/urls.py` (`/api/pagos/webhook/<pasarela>/`)

Qué hacer

1. Sin CSRF y sin autenticación de usuario. Lee el cuerpo en crudo y llama a `verificar_webhook()`; si la firma no es válida responde 400 y guarda el evento con `firma_valida = False`.
2. Evento ya recibido → 200 sin hacer nada. `cobro.pagado` → `consultar(id_externo)`: si dice pagado y el monto coincide, pendiente → pagada → acreditada; si no coincide, se crea un `Alerta` "pago inconsistente" y la orden no se acredita.

Criterios de aceptación

- ☐ Tests: firma falsa, marca de tiempo vieja (> 300 s), webhook duplicado, monto distinto y webhook de una orden inexistente.

M11-05 Expiración y reconsulta de órdenes pendientes instancia 1 servidor

Depende de: M11-02 · **Archivos:** `pagos/management/commands/revisar_ordenes.py`

Qué hacer

1. Cada 5 min (cron de la VM): las órdenes pendientes con más de 2 min se consultan a la pasarela (por si se perdió el webhook) y se acreditan o se marcan fallidas; las vencidas pasan a expirada.

Criterios de aceptación

- ☐ Con "no enviar webhook" en el simulador, la orden igual se acredita en ≤ 5 min.

M11-06 Conciliación diaria instancia 1 servidor

Depende de: M11-02 · **Archivos:** `pagos/management/commands/conciliar_pagos.py`

Qué hacer

1. `conciliar_pagos --dia AAAA-MM-DD` (ayer por defecto): compara `listar_transacciones(dia)` con las órdenes. Cada diferencia (pagada en la pasarela pero no acreditada, acreditada sin transacción, montos distintos) crea un `Alerta`. Resumen en la salida.

Criterios de aceptación

- ☐ Hay un test con cada tipo de diferencia.

M11-07 Rutas de recargas de C4 instancia 1 servidor

Depende de: M11-02 · **Archivos:** `pagos/api_cliente.py` (las monta M12-02 bajo `mi.subepe.app/api/v1/`)

Qué hacer

1. `POST/GET recargas/ordenes/` y `GET recargas/ordenes/{id}/` exactamente como C4, incluidos los códigos de error. Requieren un usuario cliente (M12); mientras M12 no esté listo, los tests usan un usuario con un `Cliente` de prueba.

Criterios de aceptación

- ☐ Las respuestas validan contra el OpenAPI de M0-04 (test automático con el esquema).

M11-08 Reembolsos (solo operadores) instancia 1 servidor

Depende de: M11-02, M10-02 · **Archivos:** `pagos/servicios.py`

Qué hacer

1. `reembolsar(orden, operador, motivo)`: solo si la recarga **aún no se aplicó en la tarjeta** (`Recarga.aplicada_en` nulo). Hace el reembolso en la pasarela, revierte el asiento y registra un CambioLista(2, aplicada) para quitarla de los buses. Si ya se aplicó, se rechaza con el mensaje "requiere ajuste manual".

Criterios de aceptación

- ☐ Tests de los dos casos. El libro cuadra después del reembolso.

M12. Identidad del cliente

> **Épica M12.** Usuarios cliente separados de los operadores, con activación bancaria, OTP por correo y protección contra fuerza bruta. Incluye el ruteo de `mi.subepe.app`.

M12-01 App `clientes` y modelos instancia 1 servidor

Depende de: — · **Archivos:** nueva app `servidor_api/clientes/`

Qué hacer

1. `Cliente`: `usuario` (OneToOne a User; `username = "cli-<documento>"`, para no chocar con los operadores), `cuenta` (OneToOne a `transporte.Cuenta`), `correo`, `correo_verificado`, `creado`.
2. `CodigoActivacion`: `tarjeta` (FK), `hash` (nunca el código en claro), `expira` (+72 h), `usado_en`, `intentos`. `CodigoOTP`: `cliente` o `token_activacion`, `proposito` (activacion, inicio, correo), `hash`, `expira` (+10 min), `intentos` (máximo 5), `usado`.
3. `Dispositivo`: `cliente`, `huella` (hash del identificador de la cookie), `nombre` (resumen del user-agent), `ultimo_uso`, `confiable_hasta`. `IntentoAcceso`: `documento`, `ip`, `exito`, `fecha`, `motivo`.
4. Función `numero_impreso(tarjeta)`: `f"{tarjeta.id:08d}"` + dígito de control Luhn (9 dígitos, mostrados como `0000 0123 4`).

Criterios de aceptación

- ☐ Ningún código ni OTP se guarda en claro (test que revisa la base de datos).

M12-02 Subdominio `mi.subepe.app` y ruteo por host instancia 1 servidor

Depende de: — · **Archivos:** `config/settings.py`, `config/middleware.py` (nuevo), `portal/urls.py`, `Caddyfile`, `.env.example`, `README.md`

Qué hacer

1. Middleware: si `request.get_host()` empieza con `mi.`, usar `request.urlconf = "portal.urls"`, que incluye las páginas del portal, `api/v1/` (clientes y pagos) y `t` (SUN).
2. Ajustes: `ALLOWED_HOSTS` y `CSRF_TRUSTED_ORIGINS` con los dos hosts, tomados de `.env` (`DOMINIO_CLIENTE`). Las cookies **sin** `SESSION_COOKIE_DOMAIN`, para que cada host tenga la suya.
3. `Caddyfile`: `{$DOMINIO}, {$DOMINIO_CLIENTE} { ... }`. Registro A `mi` en Name.com hacia la IP de la VM (lo hace el dueño del dominio). Para desarrollo local: `mi.localhost:8000`.
4. Un operador con sesión en `subepe.app` **no** tiene sesión en `mi.subepe.app`, y al revés.

Criterios de aceptación

- ☐ Tests con `HTTP_HOST` de los dos dominios: las rutas del panel no existen en `mi.` y las del portal no existen en el dominio principal.

M12-03 Activación en 3 pasos instancia 1 servidor

Depende de: M12-01, M13-01 · **Archivos:** `clientes/api.py`, `clientes/servicios.py`

Qué hacer

1. Las 3 rutas de activación de C4. Paso 1: documento + número impreso (con Luhn válido) + código, contra el `CodigoActivacion` vigente de esa tarjeta; el error es el mismo mensaje genérico para cualquier dato incorrecto. Tras 5 fallos por documento, bloqueo por 30 min.
2. El `token_activacion` se firma con `django.core.signing` y dura 15 min.
3. Paso 3: contraseña con los validadores de Django y una longitud mínima de 10; crea el `User` y el `Cliente`, marca el código como usado, inicia sesión y registra el dispositivo como confiable.

Criterios de aceptación

- ☐ Tests: código vencido, ya usado, Luhn inválido, 5 fallos → 429, y cuenta que ya tiene cliente → error.

M12-04 Inicio de sesión con OTP en dispositivo nuevo y límite de intentos instancia 1 servidor

Depende de: M12-01, M13-01 · **Archivos:** `clientes/api.py`, `clientes/servicios.py`

Qué hacer

1. Rutas de sesión de C4. Cookie `sp_disp` (aleatoria, firmada, HttpOnly, Secure, SameSite=Lax, 1 año). Si no corresponde a un `Dispositivo` confiable, se envía un OTP y la sesión queda a medias.
2. Límites: 5 fallos por documento en 15 min → `bloqueo_temporal` de 15 min; 20 fallos por IP en 15 min → 429. Todo intento se registra en `IntentoAcceso`.
3. Sesión: expira tras 15 min de inactividad (`set_expiry(900)` en cada petición del portal), y se rota la clave de sesión al iniciar.
4. Un inicio desde un dispositivo nuevo dispara la notificación "nuevo inicio de sesión" (M13).

Criterios de aceptación

- ☐ Tests de cada límite, OTP incorrecto 5 veces → OTP anulado, y "recordar dispositivo".

M12-05 Recuperación y cambio de contraseña instancia 1 servidor

Depende de: M12-01, M13-01 · **Archivos:** `clientes/api.py`

Qué hacer

1. Rutas de contraseña de C4 con `PasswordResetTokenGenerator`; el enlace lleva a `https://mi.subepe.app/restablecer?uid=..&token=..`. "Recuperar" responde siempre 202, exista o no el documento.
2. Al restablecer o cambiar: cerrar las demás sesiones del usuario y avisar por correo.

Criterios de aceptación

- ☐ Tests: el token sirve una sola vez, un token vencido falla y la respuesta no revela si el documento existe.

M12-06 Rutas de consulta (resumen, movimientos, perfil, seguridad) instancia 1 servidor

Depende de: M12-01 · **Archivos:** `clientes/api.py`, `clientes/serializers.py`

Qué hacer

1. Las rutas de consulta de C4. `documento_enmascarado` = `"****" + últimos 4`. Un cliente **solo** ve su propia cuenta; cualquier `id` ajeno responde 404.
2. Los movimientos salen de `transporte.Movimiento`; el campo `descripcion` se construye con el tipo y el validador ("Viaje · bus V103").

Criterios de aceptación

- ☐ Hay tests de aislamiento entre dos clientes y las respuestas validan contra el OpenAPI.

M12-07 Bloqueo de tarjeta y solicitud de reposición instancia 1 servidor

Depende de: M12-01 · **Archivos:** `clientes/api.py`, modelo `clientes.SolicitudReposicion`

Qué hacer

1. `POST tarjetas/{id}/bloquear/`: vuelve a pedir la contraseña y llama a `servicios.bloquear(tarjeta, "reportada por el cliente: <motivo>", tipo="bloqueo por cliente")`, que ya registra el CambioLista urgente si M9-02 está hecho.
2. `SolicitudReposicion`: `tarjeta`, `estado` (solicitada, lista, entregada), `creada`, `atendida_por`.

Criterios de aceptación

- ☐ Tests: contraseña incorrecta, tarjeta ajena (404), tarjeta ya bloqueada (409) y bloqueo correcto (estado y alerta).

M12-08 Endurecimiento del portal instancia 1 servidor

Depende de: M12-02 · **Archivos:** `config/middleware.py`, `settings.py`

Qué hacer

1. Cabeceras en `mi.`: `Content-Security-Policy: default-src 'self'; frame-ancestors 'none'`, `Referrer-Policy: same-origin`, `X-Content-Type-Options: nosniff`, HSTS (con `DJANGO_HTTPS=1`).
2. Las rutas de la API responden siempre JSON, también en 404 y 500.

Criterios de aceptación

- ☐ Hay un test que comprueba las cabeceras y la nota de securityheaders.com es A o mejor (captura en el PR).

M13. Notificaciones por correo

> **Épica M13.** Servicio de envío con bandeja de salida y reintentos, usando Gmail SMTP. Ninguna operación de dinero falla porque falle el correo.

M13-01 App `notificaciones`, bandeja de salida y `notificar()` instancia 1 servidor

Depende de: — · **Archivos:** nueva app `servidor_api/notificaciones/`

Qué hacer

1. Modelo `Notificacion`: `destinatario` (correo), `tipo`, `asunto`, `texto`, `html`, `estado` (pendiente, enviada, fallida), `intentos`, `creada`, `enviada_en`, `error`.
2. `notificar(destinatario, tipo, **contexto)`: renderiza `notificaciones/<tipo>.txt` y `.html`, la guarda e intenta enviarla en `transaction.on_commit`. Los OTP se envían al instante; el resto, en orden.

Criterios de aceptación

- ☐ Tests con el backend `locmem`: si la transacción se revierte no se envía nada; si el SMTP falla, la notificación queda pendiente con el error.

M13-02 Configuración de Gmail SMTP instancia 1 servidor

Depende de: M13-01 · **Archivos:** `config/settings.py` (`MAILERS`), `.env.example`, `README.md`

Qué hacer

1. Si `.env` tiene `CORREO_USUARIO` y `CORREO_CLAVE_APP`: backend SMTP `smtp.gmail.com:587` con STARTTLS, configurado según el ajuste `MAILERS` de Django 6.1 (comprobar los nombres de las opciones en su documentación). Si no, sigue el backend de consola. `DEFAULT_FROM_EMAIL = "SubePe <cuenta@gmail.com>"`.
2. Documentar cómo crear la contraseña de aplicación (requiere la verificación en 2 pasos de Google) y el límite de unos 500 correos al día.

Criterios de aceptación

- ☐ `manage.py sendtestemail` llega a una bandeja real, desde la VM.

M13-03 Plantillas y disparadores instancia 1 servidor

Depende de: M13-01 · **Archivos:** `notificaciones/templates/notificaciones/`, `notificaciones/disparadores.py`

Qué hacer

1. Tipos y plantillas: `otp`, `activacion_completa`, `nuevo_inicio_sesion`, `contrasena_cambiada`, `recarga_acreditada`, `tarjeta_bloqueada`, `uso_tarjeta_bloqueada`, `saldo_bajo`, `reposicion_lista`, `correo_cambiado` (al correo anterior).
2. Funciones `al_acreditar(orden)`, `al_bloquear(tarjeta)`, `al_registrar_viaje(movimiento)` (saldo bajo = saldo < 2 × tarifa, como máximo 1 al día; uso de tarjeta bloqueada si el movimiento dispara ese `Alerta`). Las conectan los dueños de cada módulo (M11, M12, M17), y si el cliente no tiene correo verificado no hacen nada.
3. HTML simple sin imágenes externas y con versión en texto plano.

Criterios de aceptación

- ☐ Hay un test de renderizado por plantilla y se adjuntan capturas en Gmail web y móvil.

M13-04 Reintentos (`enviar_notificaciones`) instancia 1 servidor

Depende de: M13-01 · **Archivos:** `notificaciones/management/commands/enviar_notificaciones.py`

Qué hacer

1. Cron cada 5 min: envía las pendientes con espera creciente (5, 15, 60 y 240 min). Tras 5 intentos queda fallida y se crea un `Alerta`. Máximo 50 por ejecución.

Criterios de aceptación

- ☐ Tests de la espera y del límite.

M14. Portal del cliente (interfaz)

Épica M14. La "banca del cliente" en `mi.subepe.app`. Web pensada primero para celular, sin frameworks ni compilación (HTML + CSS + JS modular), servida por la app Django `portal`. Se construye **contra `api-simulada.js`** y no espera al servidor.

Reglas comunes de M14: diseñada para 360 px de ancho y luego para escritorio; sin CDN (la CSP solo permite `'self'`); textos en español claro; montos como `S/ 12.40` (formatear desde céntimos con una sola función, `soles()`); estados de carga, vacío y error en cada pantalla; accesible por teclado y con etiquetas en los formularios; contraste AA.

M14-01 Base: estructura, estilos, navegación y sesión instancia 1 portal

Depende de: M0-04 · **Archivos:** `servidor_api/portal/` (`apps.py`, `urls.py`, `views.py`, `templates/portal/base.html`, `static/portal/css/portal.css`, `static/portal/js/app.js`)

Qué hacer

1. Una plantilla base por página (sin SPA): cabecera con "SubePe", barra inferior con Inicio, Movimientos, Recargar, Mi tarjeta y Más. Variables CSS de color y espacio, con modo oscuro según el sistema.
2. `app.js`: guardia de sesión (un 401 lleva a `/ingresar`), formateo de montos y fechas, avisos (toasts) y un componente de confirmación sin `alert()`.
3. Rutas Django de las páginas: `/`, `/ingresar`, `/activar`, `/recuperar`, `/restablecer`, `/movimientos`, `/recargar`, `/recargar/resultado`, `/tarjeta`, `/seguridad`, `/perfil`.

Criterios de aceptación

- ☐ Con `?simulado=1` se navega por todas las páginas (aunque estén vacías) sin errores en la consola.

M14-02 Ingresar (con OTP) instancia 1 portal

Depende de: M14-01 · **Archivos:** `templates/portal/ingresar.html`, `static/portal/js/ingresar.js`

Qué hacer

1. Formulario: DNI (8 dígitos, teclado numérico) y contraseña (con botón para mostrarla). Si responde `requiere_otp`, se pasa a la pantalla del OTP: 6 dígitos, casilla "Recordar este dispositivo", "Reenviar código" habilitado a los 60 s.
2. Errores: `credenciales_invalidas` → "DNI o contraseña incorrectos"; `bloqueo_temporal` → cuenta regresiva con `reintentar_en`.
3. Enlaces a "Activar mi tarjeta" y "Olvidé mi contraseña".

Criterios de aceptación

- ☐ Cada código de error de C4 de estas rutas se ve bien (probado con `?error=`).

M14-03 Activar mi tarjeta (3 pasos) instancia 1 portal

Depende de: M14-01 · **Archivos:** `activar.html`, `activar.js`

Qué hacer

1. Paso 1: DNI, número de tarjeta (9 dígitos con máscara `0000 0000 0` y validación Luhn en el navegador) y código del comprobante (8 caracteres en mayúsculas, con máscara `XXXX-XXXX`). Paso 2: correo y luego el OTP. Paso 3: contraseña, con indicador de fortaleza y las reglas visibles.
2. Indicador de progreso "Paso 1 de 3". Si se recarga la página se vuelve al paso 1 (el token no se guarda en `localStorage`).

Criterios de aceptación

- ☐ El flujo completo en modo simulado termina en Inicio.

M14-04 Inicio (saldo) instancia 1 portal

Depende de: M14-01 · **Archivos:** `inicio.html`, `inicio.js`

Qué hacer

1. Tarjeta visual con el saldo grande, el nombre de la tarifa y el precio del pasaje, y "≈ N viajes" (saldo ÷ precio). Si `por_cargar_en_tarjeta` > 0: aviso "S/ X se cargarán en tu tarjeta en tu próximo viaje".
2. Estado de la tarjeta (activa o bloqueada) con su número enmascarado, los últimos 5 movimientos y los botones "Recargar" y "Bloquear tarjeta".

Criterios de aceptación

- ☐ Se prueban los estados con saldo 0, saldo negativo, tarjeta bloqueada y recarga por cargar.

M14-05 Movimientos instancia 1 portal

Depende de: M14-01 · **Archivos:** `movimientos.html`, `movimientos.js`

Qué hacer

1. Lista agrupada por día, con icono por tipo, monto en verde (+) o rojo (−) y saldo final. Filtros por tipo y rango de fechas; botón "Cargar más" (paginación de C4).

Criterios de aceptación

- ☐ Con los 45 movimientos simulados, la paginación y los filtros funcionan, y la lista vacía muestra un mensaje.

M14-06 Recargar y resultado instancia 1 portal

Depende de: M14-01 · **Archivos:** `recargar.html`, `recargar.js`, `resultado.html`, `resultado.js`

Qué hacer

1. Montos rápidos (S/ 5, 10, 20, 50) y un campo libre (S/ 1.00–200.00), con un resumen antes de confirmar. Se genera un UUID v4 para `Idempotency-Key` **una vez** por intento de compra y se reutiliza en los reintentos (doble clic, red caída).
2. Se redirige a `url_pago`. Al volver (`/recargar/resultado?orden=`) se consulta la orden cada 2 s durante 60 s como máximo: acreditada → éxito; fallida o expirada → error con "Intentar de nuevo"; pendiente pasado el tiempo → "Estamos confirmando tu pago, te avisaremos por correo".

Criterios de aceptación

- ☐ El doble clic en "Pagar" crea una sola orden (se ve en el modo simulado), y los 3 resultados se muestran bien.

M14-07 Mi tarjeta: bloqueo y reposición instancia 1 portal

Depende de: M14-01 · **Archivos:** `tarjeta.html`, `tarjeta.js`

Qué hacer

1. Datos de la tarjeta (número enmascarado, emisión, último uso, estado). Botón rojo "Bloquear tarjeta": elegir el motivo (robo o pérdida), advertencia "Esta acción es definitiva. Tu saldo se conserva y pasará a tu nueva tarjeta", y la contraseña para confirmar.
2. Con la tarjeta bloqueada: botón "Solicitar reposición", estado de la solicitud y las instrucciones para recogerla en ventanilla con el DNI.

Criterios de aceptación

- ☐ El bloqueo se hace en ≤ 3 toques desde Inicio y se ven los estados solicitada, lista y entregada.

M14-08 Seguridad y perfil instancia 1 portal

Depende de: M14-01 · **Archivos:** `seguridad.html`, `perfil.html` y sus JS, `recuperar.html`, `restablecer.html`

Qué hacer

1. Seguridad: cambiar la contraseña, los últimos accesos (los fallidos resaltados), los dispositivos confiables con un botón "Quitar" y "Cerrar sesión".
2. Perfil: nombre, DNI enmascarado y correo, con el cambio de correo (y su OTP).
3. Recuperar y restablecer contraseña.

Criterios de aceptación

- ☐ Todos los flujos funcionan en modo simulado, con sus errores.

M14-09 Revisión de calidad instancia 1 portal

Depende de: M14-02 … M14-08 · **Archivos:** —

Qué hacer

1. Pasar Lighthouse (móvil) en Inicio, Movimientos y Recargar. Probar en Chrome para Android y Safari para iOS reales y en 360, 390 y 1280 px de ancho.

Criterios de aceptación

- ☐ Lighthouse: Accesibilidad ≥ 95, Buenas prácticas ≥ 95 y Rendimiento ≥ 90 (capturas en el PR).

13. Instancia 2: módulos dependientes

Se reparten igual que los de la instancia 1 y se pueden empezar con simuladores, pero **se cierran después** de sus dependencias.

M15. Gestor de enlaces

> **Épica M15.** Conecta M4 con M5, M6, M7 y M8 aplicando las reglas de C2. Es lo que hace que los canales no se solapen.

M15-01 `CanalMqtt` sobre cualquier `Enlace` instancia 2 firmware

Depende de: M0-03, M5-03 o M6-01 · **Archivos:** `lib/Enlaces/CanalMqtt.h/.cpp`, `platformio.ini` (+ `knolleary/PubSubClient@^2.8`)

Qué hacer

1. Sobre `Enlace::abrir(servidor, 8883)`: conectar con usuario = n.º de validador y la clave MQTT de la NVS, *last will* en `sp/v/{n}/conexion`, suscripción a `sp/v/{n}/baja` y `sp/todos/urgente`, y `setBufferSize(2200)`. `enviar()` publica en `sp/v/{n}/sube`.

Criterios de aceptación

- ☐ Funciona igual sobre `EnlaceWifi` y sobre `Enlace4G` (la misma clase, sin `#ifdef`).

M15-02 `GestorEnlaces` e integración en `main.cpp` instancia 2 firmware

Depende de: M15-01, M4-01, M7-03 · **Archivos:** `src/validador/gestor_enlaces.cpp/.h`, `main.cpp`

Qué hacer

1. Implementar las 5 reglas de C2: prioridad WiFi > 4G > BLE, histéresis de 10 s, cambio solo entre intercambios, 4G en reposo tras 5 min de WiFi, y ESP-NOW en paralelo.
2. Reemplazar `Sincronizador` (v1) por `SincronizadorV2` + `GestorEnlaces`. Opción `sync=v1` en `config.txt` como vuelta atrás durante un mes.

Criterios de aceptación

- ☐ El tiempo máximo de `loop()` se mantiene < 60 ms con todos los canales activos (medido y reportado).

M15-03 Estado de los canales en el LCD y en el HOLA instancia 2 firmware

Depende de: M15-02 · **Archivos:** `lib/PantallaUI/`, `gestor_enlaces.cpp`

Qué hacer

1. Icono del canal activo (W, 4G, BT o "—"), la señal y los pendientes en la barra de estado. HOLA con el enlace, la señal y el consumo.

Criterios de aceptación

- ☐ Hay fotos de cada estado en el PR.

M15-04 Pruebas de conmutación instancia 2 firmware docs

Depende de: M15-02 · **Archivos:** `docs/informe-conmutacion.md`

Qué hacer

1. Escenarios: apagar el router con 4G disponible, sacar la SIM con WiFi disponible, sin ninguno con un conductor por BLE, y volver al WiFi. Medir el tiempo de recuperación y comprobar que no hay eventos perdidos ni duplicados en el servidor.

Criterios de aceptación

- ☐ Informe con la tabla escenario / tiempo / resultado y 0 pérdidas.

M16. Relé web del conductor

> **Épica M16.** El conductor abre `https://subepe.app/conductor/` en Chrome para Android, toca "Sincronizar bus" y su celular hace de puente por BLE. No hay app.

M16-01 App `conductor`, grupo y acceso

instancia 2 · servidor

Depende de: M9-04 · **Archivos:** nueva app `servidor_api/conductor/`, migración del grupo `conductores`, panel de usuarios

Qué hacer

1. Grupo `conductores`. El superusuario lo asigna desde la gestión de usuarios del panel. Página `/conductor/` con el inicio de sesión del panel, que solo exige ese grupo (sin permisos de operador).

Criterios de aceptación

- ☐ Un conductor no puede entrar a las demás páginas del panel (test).

M16-02 Página Web Bluetooth

instancia 2 · portal

Depende de: M16-01, M7-02 · **Archivos:** `conductor/static/conductor/rele.js`, plantilla

Qué hacer

1. "Conectar validador" → `navigator.bluetooth.requestDevice({filters: [{services: [UUID]}]})` → leer **info** y mostrar "Bus V103 · 45 pendientes" → control = 1 → reensamblar los fragmentos de **salida** → `POST /api/v2/paquetes/` → escribir las respuestas fragmentadas en **entrada** → control = 3.
2. Barra de progreso y resumen final ("45 viajes subidos · listas actualizadas a v1234"). Si el navegador no tiene `navigator.bluetooth`: "Usa Chrome en Android".

Criterios de aceptación

- ☐ Ciclo completo con un validador real y un celular Android real en < 20 s con 100 eventos (medido).

M16-03 Celular sin datos: guardar y subir después

instancia 2 · portal

Depende de: M16-02 · **Archivos:** `rele.js`, `conductor/static/conductor/sw.js` (service worker)

Qué hacer

1. Si el POST falla por falta de red: guardar los paquetes en IndexedDB, mostrar "Guardados N paquetes, se subirán al tener datos" y subirlos al volver la conexión (evento `online` o al abrir la página). La página funciona sin conexión gracias al service worker.
2. Los acuses de esa subida llegan al validador en su próxima sincronización por cualquier canal; reenviar eventos no causa daño.

Criterios de aceptación

- ☐ Prueba en modo avión: se toma la cola, se activan los datos y los viajes aparecen en el servidor.

M16-04 Guía de uso para el conductor

instancia 2 · docs

Depende de: M16-02 · **Archivos:** `docs/guia-conductor.md` (y una versión imprimible de una página)

Qué hacer

1. Pasos con capturas: abrir la página, iniciar sesión, conectar y qué hacer ante cada error.

Criterios de aceptación

- ☐ Una persona que no conoce el sistema completa la sincronización solo con la guía.

M17. Robo y reposición de punta a punta

> **Épica M17.** Desde que el cliente pulsa "Bloquear" hasta que tiene una tarjeta nueva con su saldo.

M17-01 Conectar el flujo y las notificaciones instancia 2 servidor

Depende de: M12-07, M9-06, M13-03 · **Archivos:** `clientes/servicios.py`, `transporte/servicios.py`

Qué hacer

1. Bloqueo desde el portal → CambioLista urgente → difusión MQTT → `al_bloquear()` (correo). Un viaje con tarjeta bloqueada que llega después → `uso_tarjeta_bloqueada` (correo).

Criterios de aceptación

- ☐ Test de integración que recorre el flujo hasta la notificación en la bandeja de salida.

M17-02 Medir el tiempo de propagación de un bloqueo por canal instancia 2 docs

Depende de: M17-01, M15-02, M8-03 · **Archivos:** `docs/informe-propagacion-bloqueo.md`

Qué hacer

1. Medir desde el clic en el portal hasta el rechazo de la tarjeta en el validador, por WiFi, 4G, BLE (conductor) y ESP-NOW (bus vecino). 5 repeticiones cada uno.

Criterios de aceptación

- ☐ Informe con la mediana y el máximo; el objetivo es < 1 min por WiFi y 4G.

M17-03 Reposición en ventanilla instancia 2 servidor tarjeta

Depende de: M12-07, M18-01 · **Archivos:** `transporte/servicios.py` (`confirmar_emision`), `transporte/emitir_tarjeta.py`

Qué hacer

1. `emitir_tarjeta.py --cuenta N` (que ya conserva el saldo): si hay una `SolicitudReposicion` abierta, la marca como entregada, genera el código de activación de la tarjeta nueva **solo si el cliente aún no tiene usuario** (si ya lo tiene, la tarjeta nueva aparece directamente en su portal) y notifica `reposicion_lista`.

Criterios de aceptación

- ☐ Prueba manual con una NTAG215: bloquear → reponer → la tarjeta nueva tiene el saldo y el portal la muestra.

M18. Emisión estilo bancario

> **Épica M18.** La ventanilla entrega la tarjeta y un comprobante con el código de activación.

M18-01 Código de activación al confirmar la emisión instancia 2 servidor

Depende de: M12-01 · **Archivos:** `transporte/servicios.py`, `transporte/views.py` (`emision_confirmar`)

Qué hacer

1. Al confirmar una emisión de una cuenta sin `Cliente`: generar el código (8 caracteres del alfabeto `ABCDEFGHJKMNPQRSTUVWXYZ23456789`, con `secrets`), guardar solo su hash y devolverlo **una única vez** en la respuesta, con `numero_impreso`.
2. Ruta de operador `POST /api/tarjetas/{id}/codigo-activacion/` para regenerarlo (anula el anterior), con registro de quién lo hizo.

Criterios de aceptación

- ☐ Tests: el código no se puede volver a obtener y regenerarlo anula el anterior.

M18-02 **Comprobante imprimible** instancia 2 servidor

Depende de: M18-01 · **Archivos:** `panel/templates/panel/comprobante.html`, vista del panel

Qué hacer

1. Página A6/A5 lista para imprimir: nombre, DNI enmascarado, número impreso de la tarjeta, código de activación, fecha de vencimiento (72 h), la URL `mi.subepe.app/activar` con su código QR (generado en el servidor, sin CDN) y avisos de seguridad ("Nadie de SubePe te pedirá este código").

Criterios de aceptación

- ☐ La vista previa de impresión del navegador cabe en una hoja A6.

M18-03 `emitir_tarjeta.py` **muestra el comprobante** instancia 2 tarjeta

Depende de: M18-01, M18-02 · **Archivos:** `transporte/emitir_tarjeta.py`

Qué hacer

1. Al terminar la emisión, imprimir en la terminal el número y el código y abrir la URL del comprobante en el navegador. Opción `--sin-comprobante` para pruebas.

Criterios de aceptación

- ☐ Emitir con una NTAG215 real produce el comprobante y la activación en el portal funciona con él.

M19. Panel de operadores, ampliación

> **Épica M19.** Que los operadores vean y atiendan lo nuevo: pagos, canales, libro y reposiciones.

M19-01 **Órdenes de pago y conciliación** instancia 2 servidor

Depende de: M11-02, M11-06, M11-08 · **Archivos:** `panel/views.py`, `panel/urls.py`, plantillas

Qué hacer

1. Lista de órdenes con filtros (estado, fecha, cuenta), detalle con el historial y los eventos de pasarela, botón "Reembolsar" (M11-08) con motivo obligatorio y la última conciliación con sus diferencias.

Criterios de aceptación

- ☐ Solo los operadores pueden reembolsar (test).

M19-02 **Validadores: canal, conexión y consumo** instancia 2 servidor

Depende de: M9-03 · **Archivos:** `panel/templates/panel/validadores.html`, `panel/views.py`

Qué hacer

1. Columnas nuevas: último canal, en línea (por `sp/v/{n}/conexion` o por un HOLA de menos de 6 min), versión de listas frente a la actual, pendientes, consumo del día y versión de firmware. Se resaltan los validadores con más de 1 h sin sincronizar.

Criterios de aceptación

- ☐ Los datos coinciden con el último HOLA recibido.

M19-03 **Estado del libro contable** instancia 2 servidor

Depende de: M10-05 · **Archivos:** panel

Qué hacer

1. Tarjeta en el inicio del panel: última verificación, resultado y diferencias, y el saldo de las cuentas contables fijas.

Criterios de aceptación

- ☐ Un descuadre provocado aparece en rojo.

M19-04 Solicitudes de reposición instancia 2 servidor

Depende de: M12-07 · **Archivos:** panel

Qué hacer

1. Lista de solicitudes con su estado y acciones "Marcar lista" (que notifica al cliente) y "Entregada".

Criterios de aceptación

- ☐ El cambio de estado se refleja en el portal del cliente.

M20. Integración del portal

> **Épica M20.** El portal pasa de los datos simulados a la API real.

M20-01 Conectar el portal a la API real instancia 2 portal

Depende de: M14, M12, M11-07 · **Archivos:** `portal/static/portal/js/`

Qué hacer

1. Sin `?simulado=1` se usa `api.js`. Corregir cualquier diferencia encontrada **en el contrato primero** y después en el código.

Criterios de aceptación

- ☐ Se completa la lista de flujos de M20-03 en el servidor de pruebas.

M20-02 Vista SUN para el dueño de la tarjeta instancia 2 portal

Depende de: M1-08, M12-04 · **Archivos:** vista `/t`

Qué hacer

1. Si quien abre la URL SUN tiene sesión y es el dueño de la tarjeta: mostrar el saldo y el último movimiento. Si no, solo la vista pública de M1-08.

Criterios de aceptación

- ☐ Tests con el dueño, con otro cliente y sin sesión.

M20-03 Lista de flujos de punta a punta instancia 2 portal docs

Depende de: M20-01 · **Archivos:** `docs/flujos-portal.md`

Qué hacer

1. Recorrer y documentar con capturas: emisión → activación → ingreso → recarga simulada → viaje en un validador real → movimiento visible → bloqueo → reposición.

Criterios de aceptación

- ☐ Todos los pasos funcionan en `subepe.app` con una NTAG215 real.

M21. Pruebas de robustez

> **Épica M21.** Demostrar que el sistema aguanta los fallos para los que se diseñó.

M21-01 Batería de escenarios de fallo instancia 2 firmware servidor

Depende de: M15, M16, M11 · **Archivos:** `docs/informe-robustez.md`

Qué hacer

1. Ejecutar y documentar: (1) retirar la tarjeta a mitad de la escritura, 20 veces; (2) cortar la energía del validador durante la sincronización; (3) 24 h sin ningún canal y luego recuperar; (4) un bus sin red que sincroniza solo a través de otro bus; (5) un webhook duplicado y uno perdido; (6) reinicio del servidor con validadores conectados; (7) paquetes alterados inyectados por BLE.

Criterios de aceptación

- ☐ Ningún escenario pierde ni duplica dinero, y `verificar_libro` da 0 diferencias al final.

M21-02 **Prueba de carga del servidor** instancia 2 servidor

Depende de: M9-06 · **Archivos:** `herramientas/carga_mqtt.py`

Qué hacer

1. Simular 50 validadores con 1 evento cada 10 s durante 30 min contra la VM de 1 GB. Medir la CPU, la RAM y la latencia del acuse.

Criterios de aceptación

- ☐ Informe: el acuse sale en < 2 s en el percentil 95 y la VM no usa swap de forma sostenida.

14. Hardware a comprar

Elemento	Recomendación	Notas
Módulo celular	**SIMCom A7670SA** (LTE Cat-1, bandas de Sudamérica) en una placa con regulador y adaptador de niveles. Alternativa: **SIM7600SA**	La variante **SA** cubre las bandas de los operadores peruanos (B2/B4/B5/B7/B28, entre otras; confirmar en la ficha). Comprobar que la placa trae antena LTE y ranura nano/micro SIM.
Fuente para el módulo	Convertidor reductor de 12/24 V a 5 V y ≥ 2 A (para el bus) o fuente USB de 5 V y 2 A en el banco de pruebas	Los picos al transmitir llegan a unos 2 A: nunca desde el 3V3 de la ESP32.
SIM	Prepago con un paquete de datos pequeño	Con el consumo esperado (< 30 MB/mes) alcanza el paquete más barato.
Transistor NPN + resistencias	2N2222 o BC547, 1 kΩ y 10 kΩ	Solo si la placa del módulo no expone PWRKEY de forma controlable.
2.ª y 3.ª ESP32 (opcional)	ESP32-WROOM-32 DevKit, igual a la actual	Para M8 (ESP-NOW entre buses) y M2-03 (placa de pruebas para eFuses).
Tarjetas NTAG 424 DNA	Paquete pequeño (10–20) de proveedor confiable	Comprobar la firma de originalidad de NXP al recibirlas (las NTAG215 de prueba no son originales).

15. Riesgos y temas abiertos

Riesgo o tema	Impacto	Mitigación o decisión pendiente
RAM de la ESP32-WROOM-32 con WiFi + TLS + BLE + ESP-NOW	Reinicios o fallos de TLS	M7-01 lo mide primero. Si no alcanza: BLE solo cuando no hay WiFi ni 4G, o pasar a ESP32-S3 (más RAM y HMAC en eFuse).
Compatibilidad del A7670 con TinyGSM	Retraso en M5	M5-02 lo valida antes de escribir `Enlace4G`; la alternativa es el SIM7600, con mejor soporte.
La VM gratuita tiene 1 GB de RAM	Lentitud	Mosquitto (~10 MB) y el puente (~60 MB) caben; M21-02 lo confirma. Mudanza planificada para el 1–3/11.
Clave maestra en la microSD hoy	Robo de la clave	M2-01 es prioritario.
ESP-NOW solo en el canal 1	Sin intercambio entre buses mientras el WiFi está en otro canal	Configurar los routers y hotspots en el canal 1 cuando se pueda; aceptarlo como limitación.
Web Bluetooth no funciona en iOS	Conductores con iPhone no pueden hacer de relé	Aceptado. Esos conductores pueden compartir datos por hotspot (WiFi).
Límite de Gmail (~500 correos/día)	Correos retrasados	Suficiente para el proyecto; pasar a Brevo o Resend si crece.
Comportamiento de la 424 ante escrituras interrumpidas	Diseño del registro	M1-02 lo confirma en la hoja de datos; el doble registro se mantiene igual.

16. Glosario

Acuse	Paquete del servidor que confirma hasta qué evento recibió; permite borrar la cola del validador.
Ajeno (paquete)	Paquete de otro bus que un validador transporta hacia el servidor.
CambioLista / versión	Registro numerado de cada alta o baja en la lista negra o en las recargas pendientes; los validadores piden "lo que cambió desde la versión N".
ESP-NOW	Protocolo de Espressif para comunicación directa entre ESP32, sin router (unos 100–200 m).

Foto completa	Envío del estado entero de las listas cuando un validador está muy atrasado.
HMAC / ECDSA	Firmas: HMAC con una clave compartida (validador ↔ servidor); ECDSA con una clave pública que cualquiera puede verificar pero solo el servidor puede producir.
Idempotencia	Repetir una operación no cambia el resultado (reenviar un viaje o un pago no lo duplica).
MQTT / Mosquitto	Protocolo abierto de mensajería para IoT con publicación y suscripción, y su servidor libre.
OTP	Código de un solo uso enviado por correo (6 dígitos, 10 min).
SUN / SDM	Función de la NTAG 424 que añade a la URL, en cada lectura, datos cifrados y una firma CMAC que el servidor verifica.
Web Bluetooth	API del navegador (Chrome para Android) que permite a una página web hablar por BLE con un dispositivo.